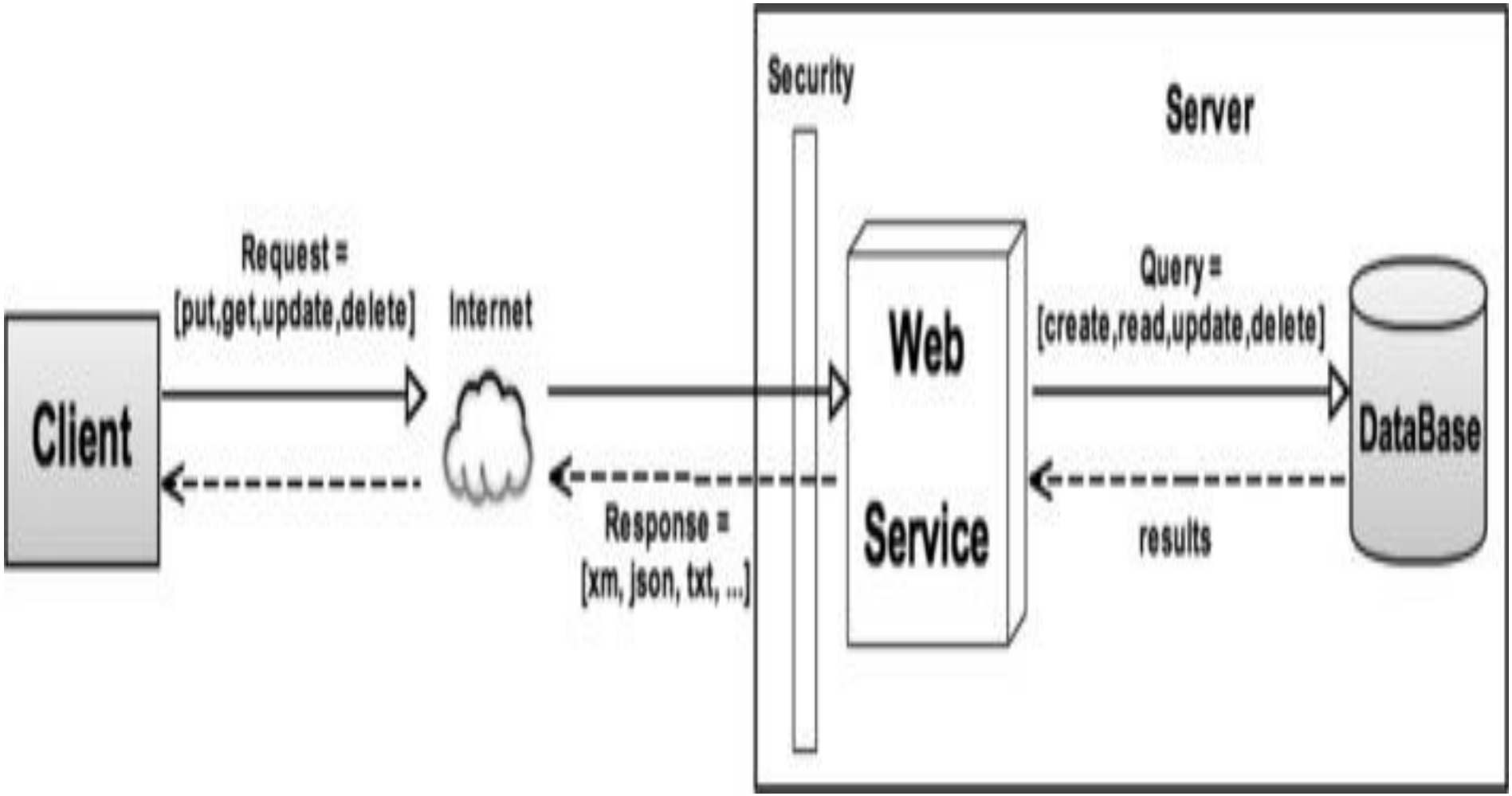
Micro Services

- Day 1

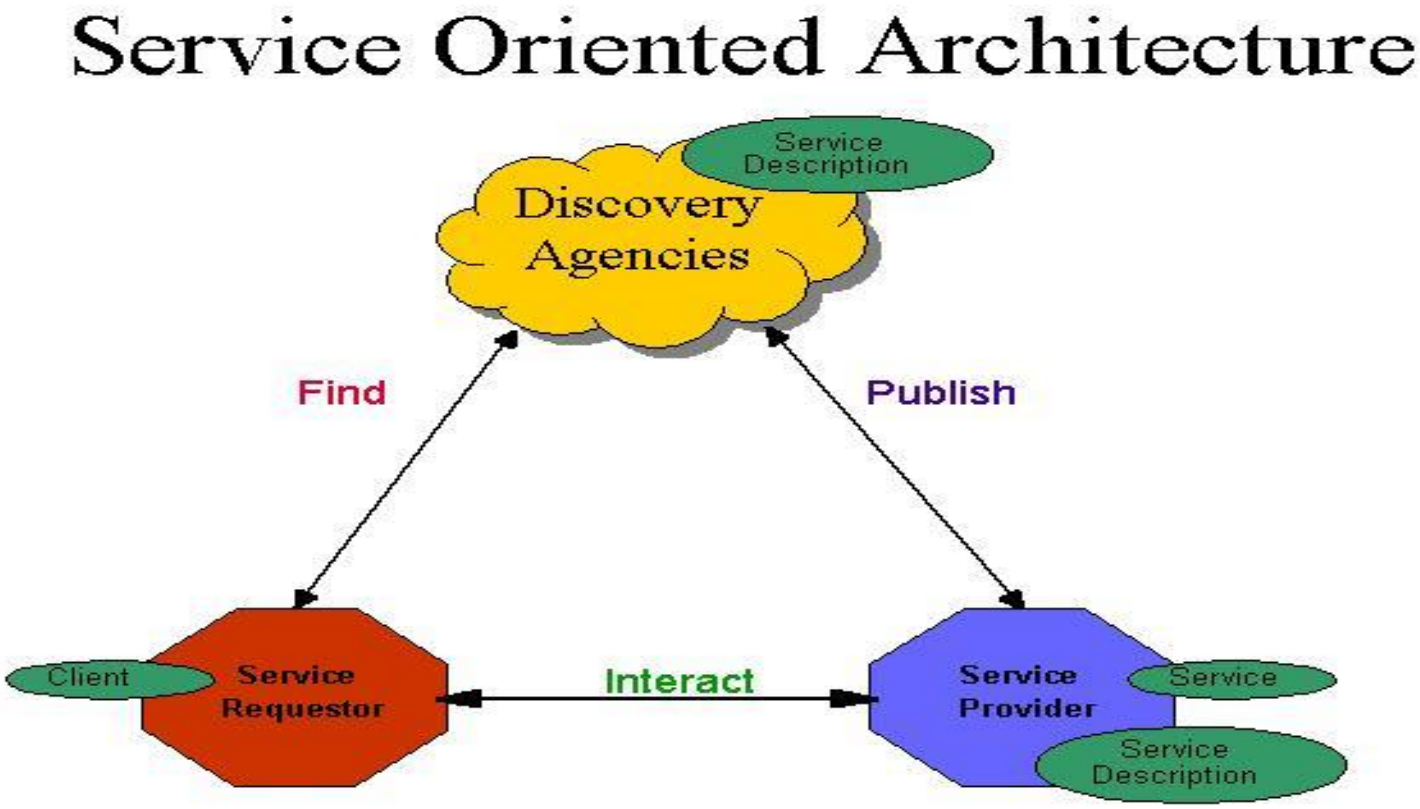
Web Services Concepts

- **Web Service** – an architectural style that defines a set of rules in order to create Web Services.
- In Client– Server communication REST suggests to create an object of the data requested by the client and send the values of the object in response to the user.
  - ✓ a technology providing for **distributed computing** to create enterprise C–S applications;
  - ✓ based on standard interface **to communicate with each** other even if they are **running on different operating systems** and written in **different languages**; XML/JSON is used for information interchange.
  - ✓ an abstract notion, a resource characterized by abstract set of **functionality** ;
  - ✓ a service which is **server–based application/software component** that exposes a resource to clients via exchange of messages according to message–exchange pattern.

contd..


Security
Server
Request =
[put,get,update,delete]
Internet
Client
Web
Service
Query =
[create,read,update,delete]
DataBase
Response =
[xm, json, txt, ...]
results

contd..

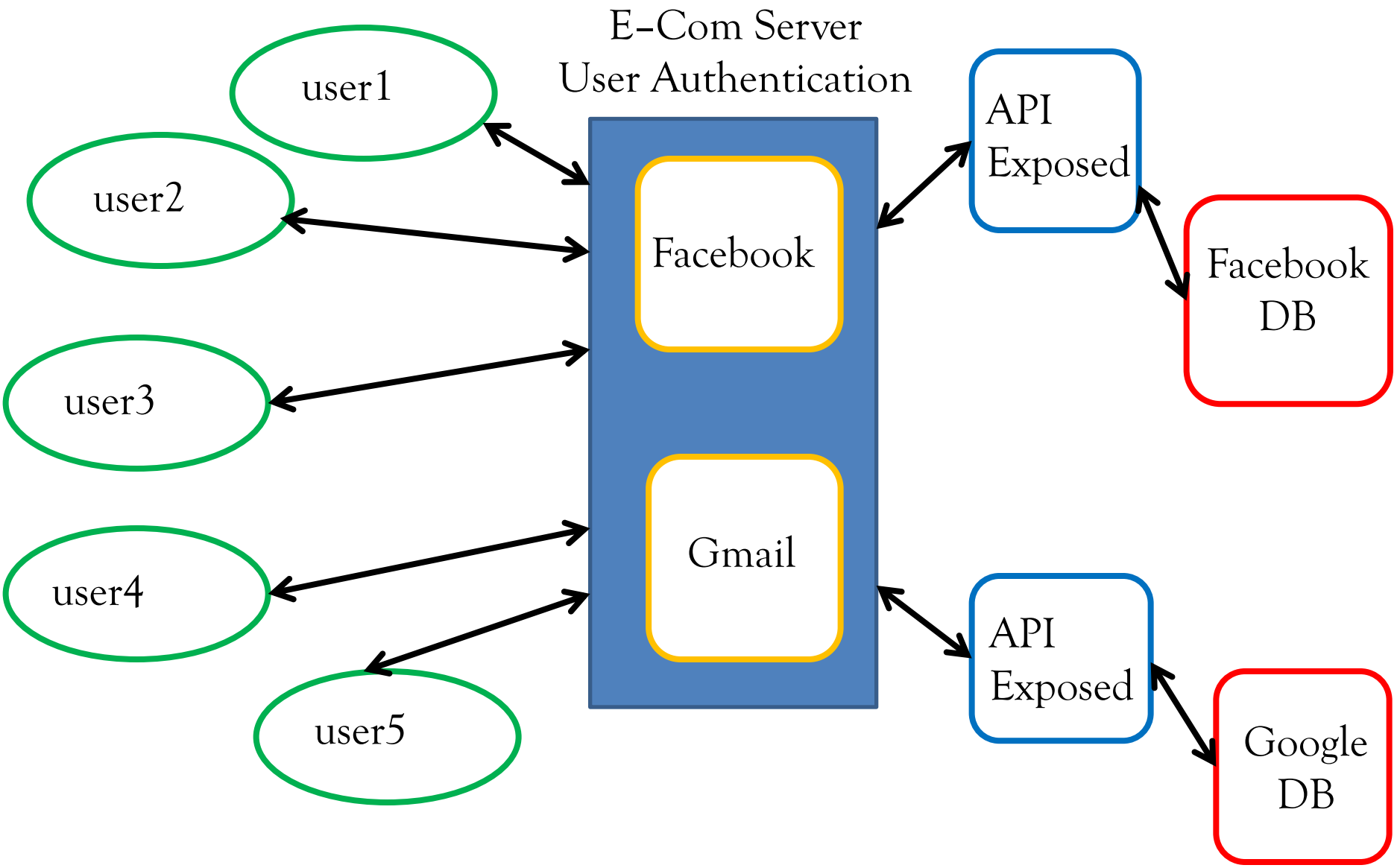

Service Oriented Architecture
Service Description
Discovery Agencies
Find
Publish
Client
Service Requestor
Interact
Service Provider
Service
Service Description

Web Services Scenario – 1

- Consider an e–Commerce application.
  - ✓ The firm decides to have in only **authenticated users** and not maintain the data of all users who login. (Meaning only those users who **use the application**).
  - ✓ Hence, use the web service exposed by some third party application to authenticate incoming users.

    E.g. Facebook, Gmail, etc.
  - ✓ Net productivity – Focus is only on the content of the application, makes software application resources available over networks using standard technologies.

contd..


E-Com Server
User Authentication
user1
user2
user3
user4
user5
Facebook
Gmail
API
Exposed
API
Exposed
Facebook
DB
Google
DB

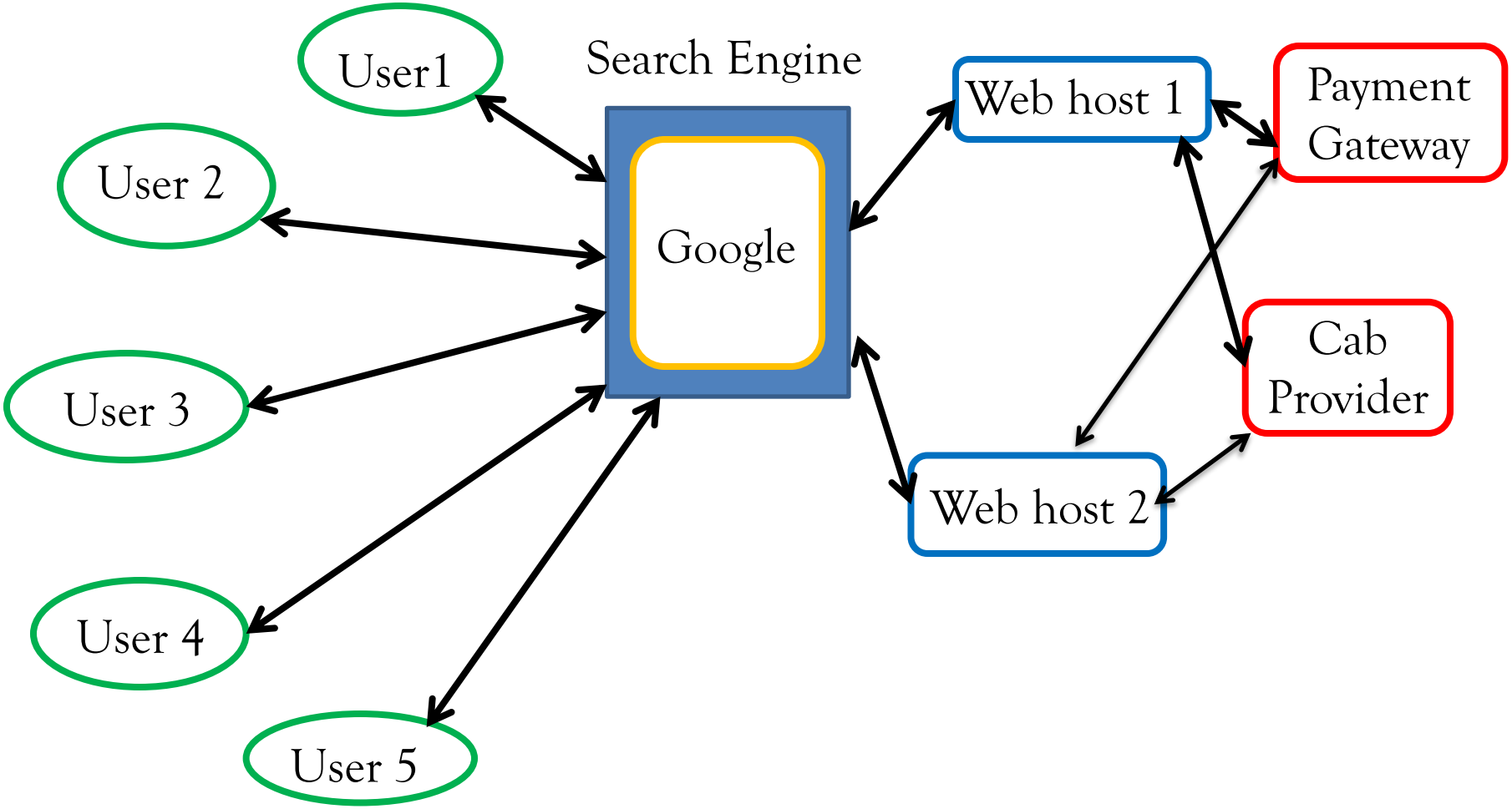
contd..

- The flow of the authentication process will be –
  1. User log into the application
  2. The application shows them two options – sign in using either Facebook or Gmail
  3. On login, the application will pass credentials to Gmail or Facebook server
  4. Step 3 is done using http request and can be in any format json/xml/media type
  5. If the request is ok, the Gmail or Facebook server will send the response to server of the firm.

Web Services Scenario – 2

- Consider some Cab provider companies. Two web – hosting companies upload their profiles.
  - ✓ The web-hosting companies provide the details of the costing, offers and cab–availability to the cab users.
  - ✓ Now, the Q is how will the cab–users know about web-hosting companies ? So, the individual web hosts register with Search Engine and register their URLs.
  - ✓ The web hosts even provide the Payment Gateway.
  - ✓ In this scenario, the search engine at level–1 and web hosts at level–2 act as Web Service providers. E.g. Google.

contd..


User1
Search Engine
Web host 1
Payment Gateway
User 2
Google
User 3
Cab Provider
Web host 2
User 4
User 5

contd..

- The flow of the search process will be –
- ✓ User searches for the cab services from Google server.
- ✓ The Google server outputs these URLs
- ✓ On selecting a single URL, the user goes on to access the app server of that particular Web host.
- ✓ Step 1 is accessing the Web Service.
- ✓ Step 3 is accessing the Web host through Web service.

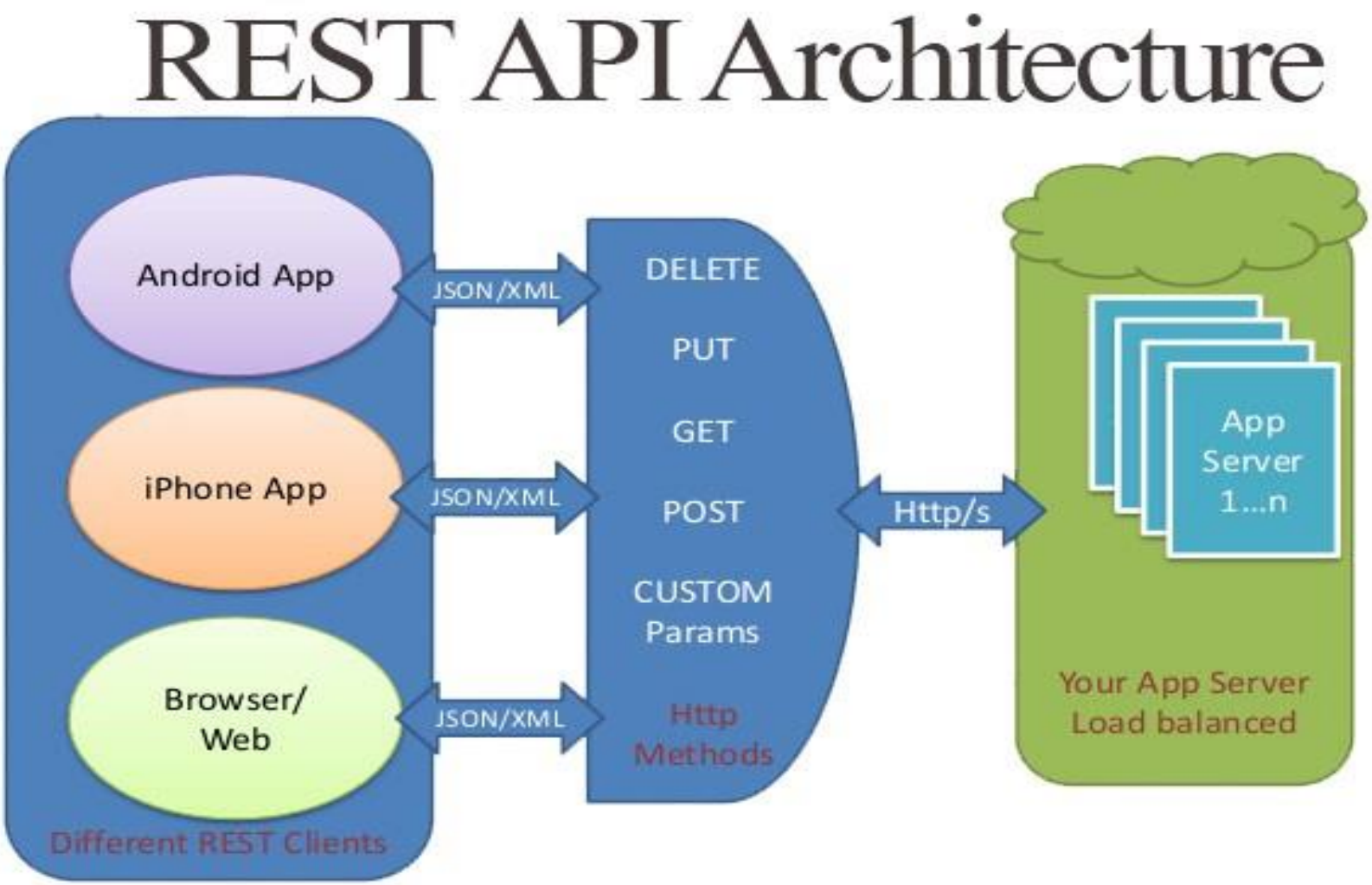
Web Services Types

- Web Services are mainly of 2 types,

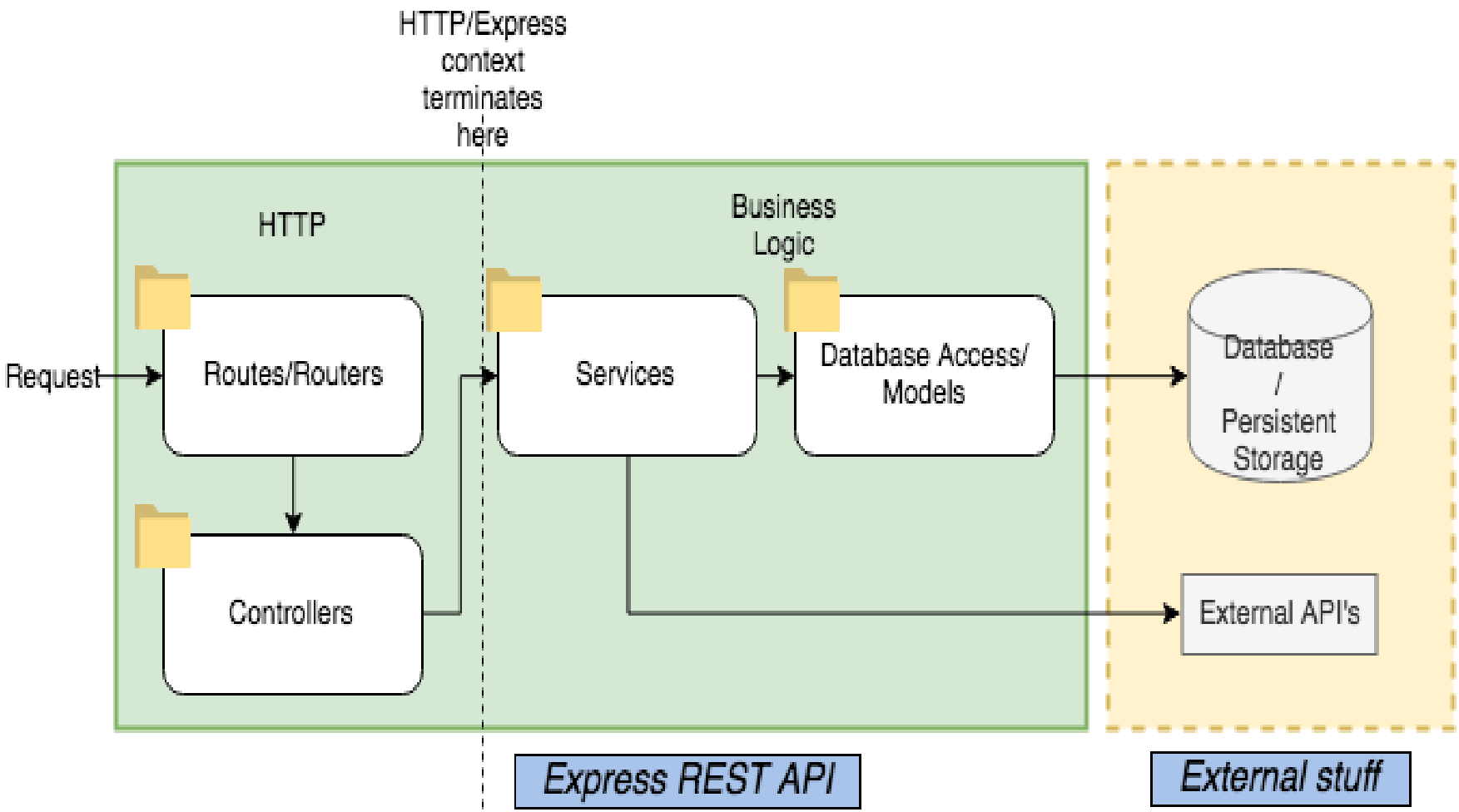
1. **SOAP** [Simple Object Access Protocol]
2. **REST** [REpresentational State Transfer] based services.

- REST - **REpresentational State Transfer** is an **architectural style,** brought out by **Roy Fielding** in his doctoral thesis in 2000, specifies constraints that induce properties such as – performance, scalability and modifiability to WS
- REST is **platform** and **protocol independent** and often associated or implemented with HTTP and also be done on other technologies.
- REST offers a **simple, interoperable** and **flexible** way of writing web services drastically different from RPC mechanisms.
- REST mainly aims to address implementation issues and organize relationship between logical clients and servers.

Top level REST API architecture



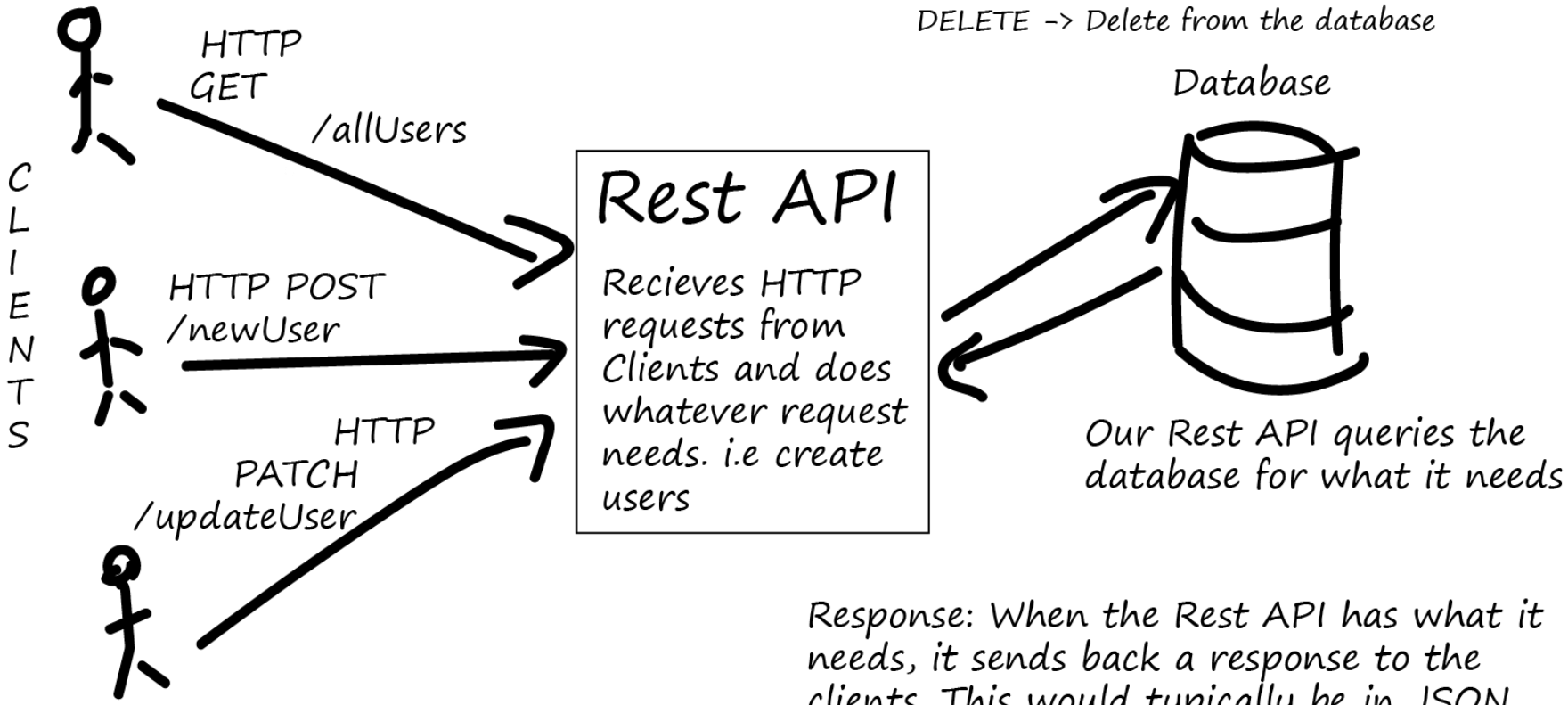
contd..


HTTP/Express
context
terminates
here
HTTP
Business
Logic
Request
Routes/Routers
Services
Database Access/
Models
Database
/
Persistent
Storage
Controllers
External API's
Express REST API
External stuff

contd..

Rest API Basics

Typical HTTP Verbs:
GET -> Read from Database
PUT -> Update/Replace row in Database
PATCH -> Update/Modify row in Database
POST -> Create a new record in the database
DELETE -> Delete from the database



Our Clients, send HTTP Requests and wait for responses

Response: When the Rest API has what it needs, it sends back a response to the clients. This would typically be in JSON or XML format.

Five Principles of REST

1. Give every "thing" an **ID** –

- Every system that is being built, has **set of abstractions** that need **to be identified** and everything that is **identifiable** should get ID.
- The **URIs** make up **global namespace**, and using **URIs** to identify key resources means they get a **unique, global ID**.
- **URIs** identify everything that merits being identifiable, even all of the "high-level" resources that application provide. viz… individual & collections of items, virtual & physical objects, computed results.
- The developers don't have to come up with new scheme, existing one can be used, works on global scale and understood by all.
- Online store identifies its products with unique ID (a URI) –
  - ✓ http://example.com/customers/1234
  - ✓ http://example.com/orders/2007/10/776654
  - ✓ http://example.com/products/4554
  - ✓ http://example.com/processes/salary-increase-234

contd..

2. **Link things together**

- "**Hypermedia As The Engine Of Application State**", sometimes abbreviated as **HATEOAS**. The core concept is of ***hypermedia,*** or the idea of ***links***. Links are not restricted to human consumption.
- The **naming scheme** is a **global standard**, all of the **resources** that make up the Web can be linked to each other.
- **Links** can point to **resources** that are **provided** by a **different application, different server,** or **different company** on another continent.
- The **server (or service provider)** provides a **set of links** to the **client (the service consumer)** enabling the client to move the application from one state to the next by following a link.

contd..

3. **Use standard methods –**

- Every resource **supports** the **same interface**, the same set of **methods** or **operations**.
- HTTP calls them ***verbs***, and in addition to the two (GET and POST), the set of standard methods includes PUT, DELETE, HEAD and OPTIONS.
- The meaning of these methods is defined in the HTTP specification, along with some guarantees about their behavior.
- E.g. an OO developer can imagine that every resource in a RESTful HTTP scenario extends a class like this (in some Java/C# style pseudo–syntax and concentrating on the key methods):

contd..

4. Resources with multiple representations

1. Everything is **data**

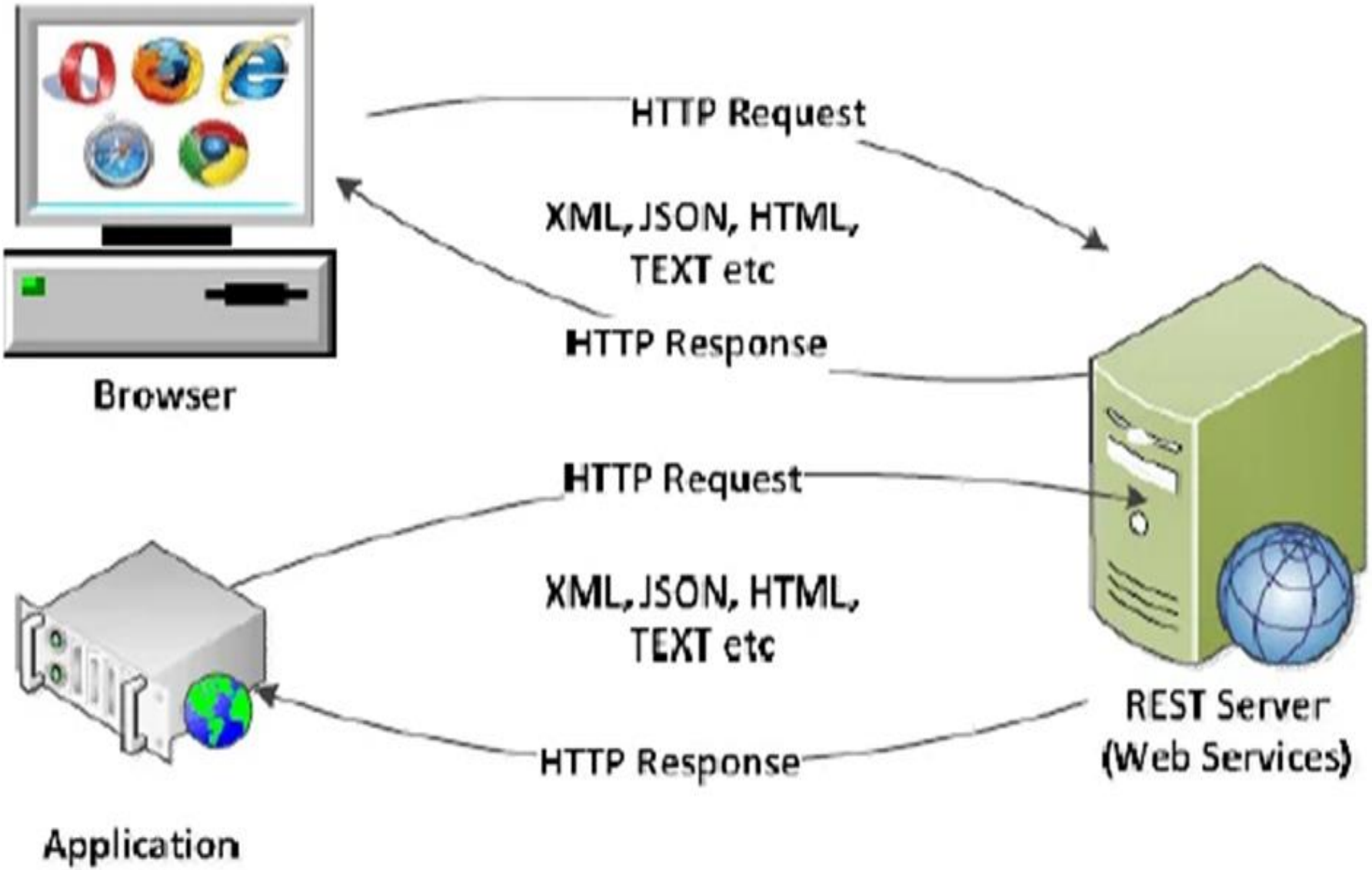
- **Data** is called **resource**. Every resource has **unique addressable location** and is represented by **web address**.
- There is **one endpoint** for each resource and the **endpoint** says nothing about how the data is represented
- REST is a set of principles that define how Web standards, such as HTTP and URIs, are supposed to be used, for designing loosely coupled applications used in the development of web services.
- REST does not enforce any rule regarding how it should be implemented at lower level, it just put high level design guidelines and allows developer to think of your own implementation.

contd..

5. **Communicate statelessly**

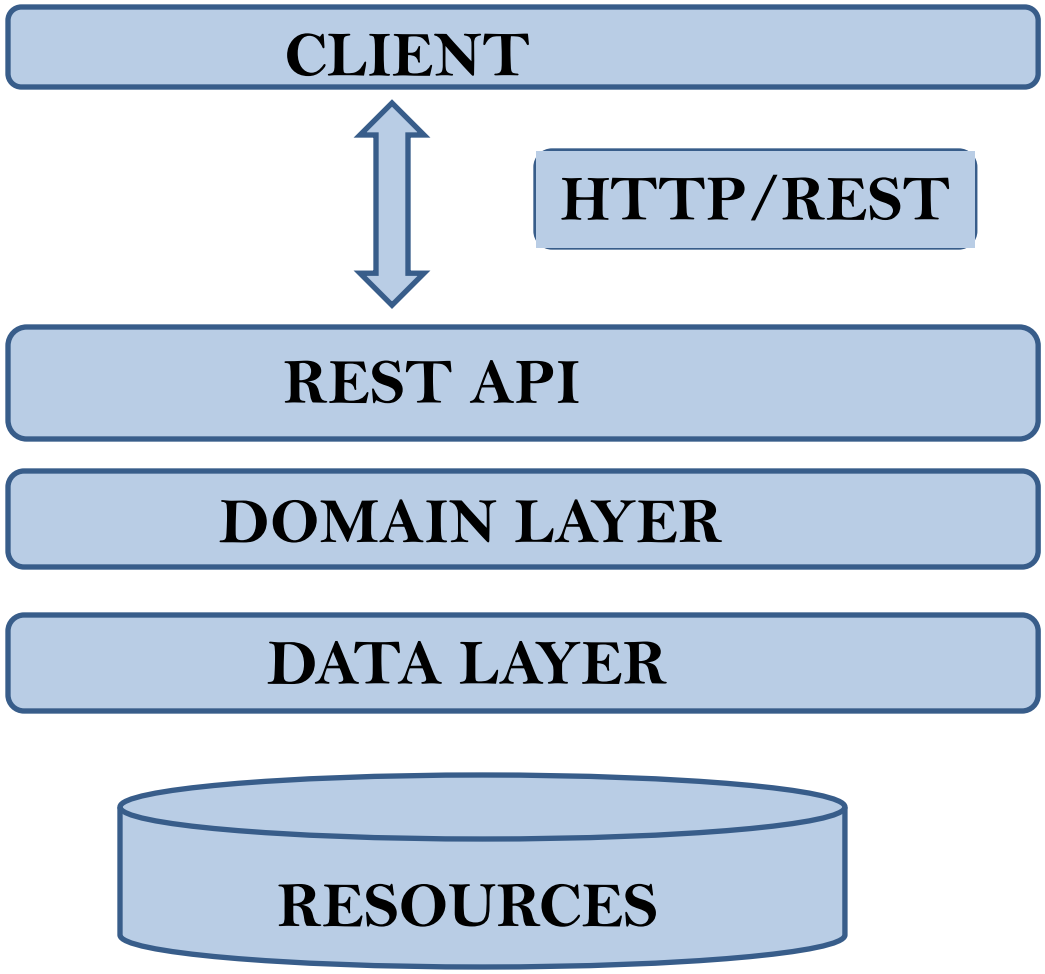
- REST mandates that **state** be either turned into **resource state**, or kept on the client, i.e. a server should not have to retain some sort of **communication state** for any of the clients it communicates with beyond a single request.
- Although REST includes the idea of **statelessness**, doesn't mean that an application exposing functionality, cannot have state
- The statelessness constraint **isolates** the client **against changes** on **the server** as it is not dependent on talking to the same server in two consecutive requests.
- E.g. client could receive a document containing links from the server, and when processing the server could be shut down or could be ripped out and be replaced — and if the client follows one of the links it has received from the server, it won't notice.

contd..


HTTP Request
XML, JSON, HTML,
TEXT etc
HTTP Response
Browser
HTTP Request
XML, JSON, HTML,
TEXT etc
HTTP Response
REST Server
(Web Services)
Application

Logical Layers of REST

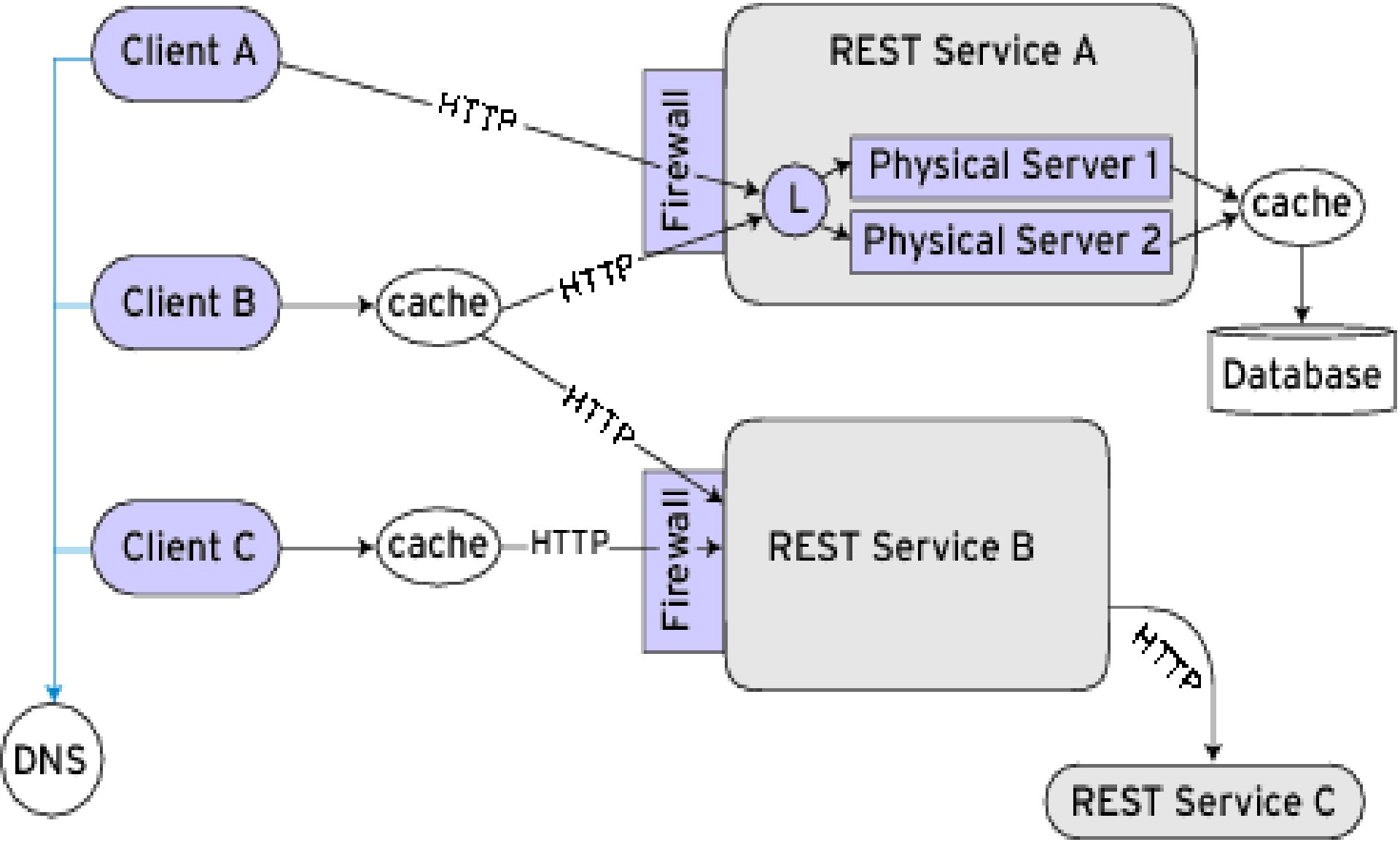
- There are four layers to the REST application



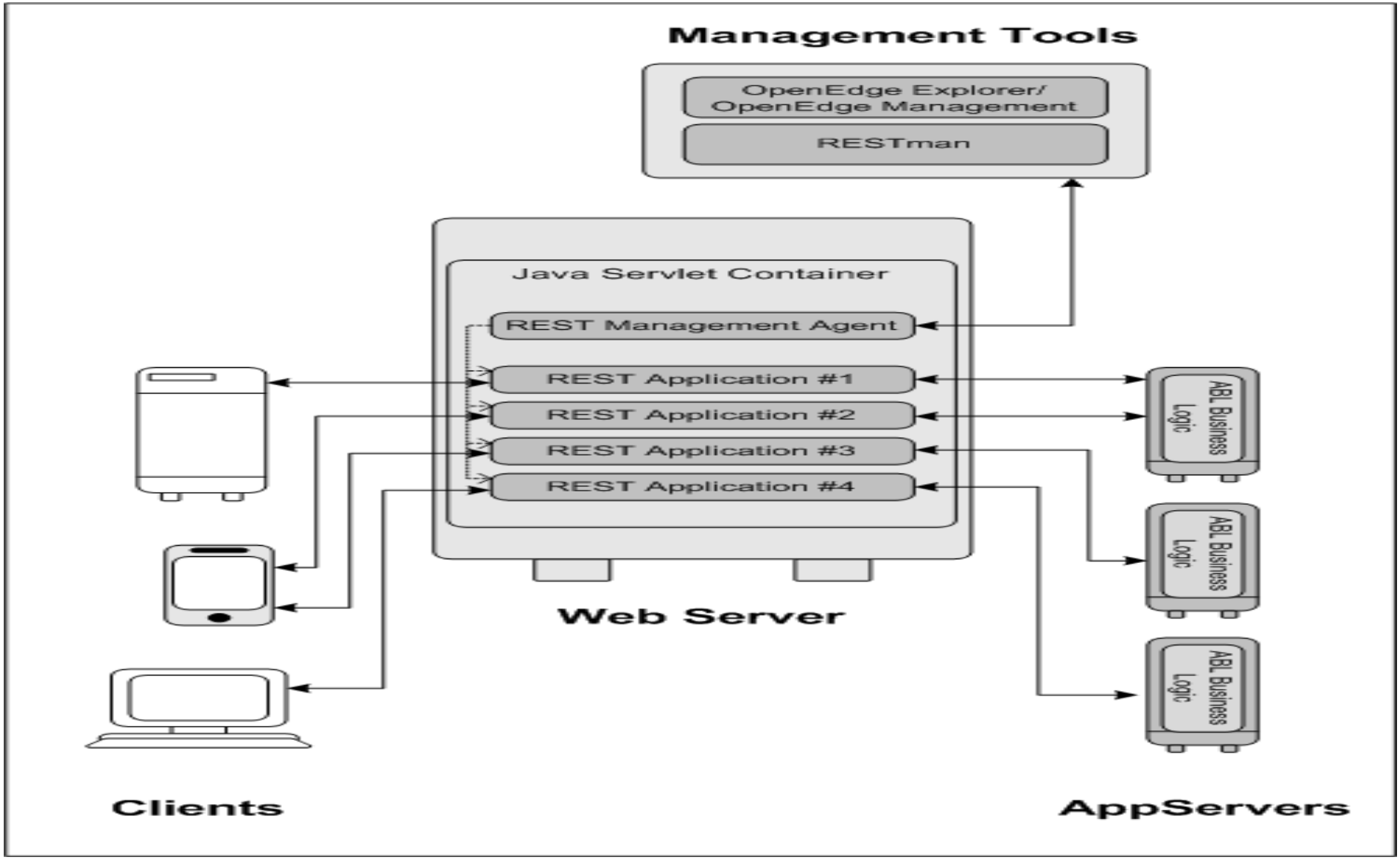
contd..

- REST aims to achieve –
  - ✓ Generality of Interface,
  - ✓ Independent Deployment of Components
  - ✓ Encapsulate Legacy Systems and Enforce Security, Emphasizing the scalability of Component Interactions
- **Architectural Constraints –** REST defines **6 architectural constraints** which make any web service – a true RESTful API.
  - √ Uniform Interface
  - √ Client – server
  - √ Stateless
  - √ Cacheable
  - √ Layered system
  - √ Code on demand (optional)

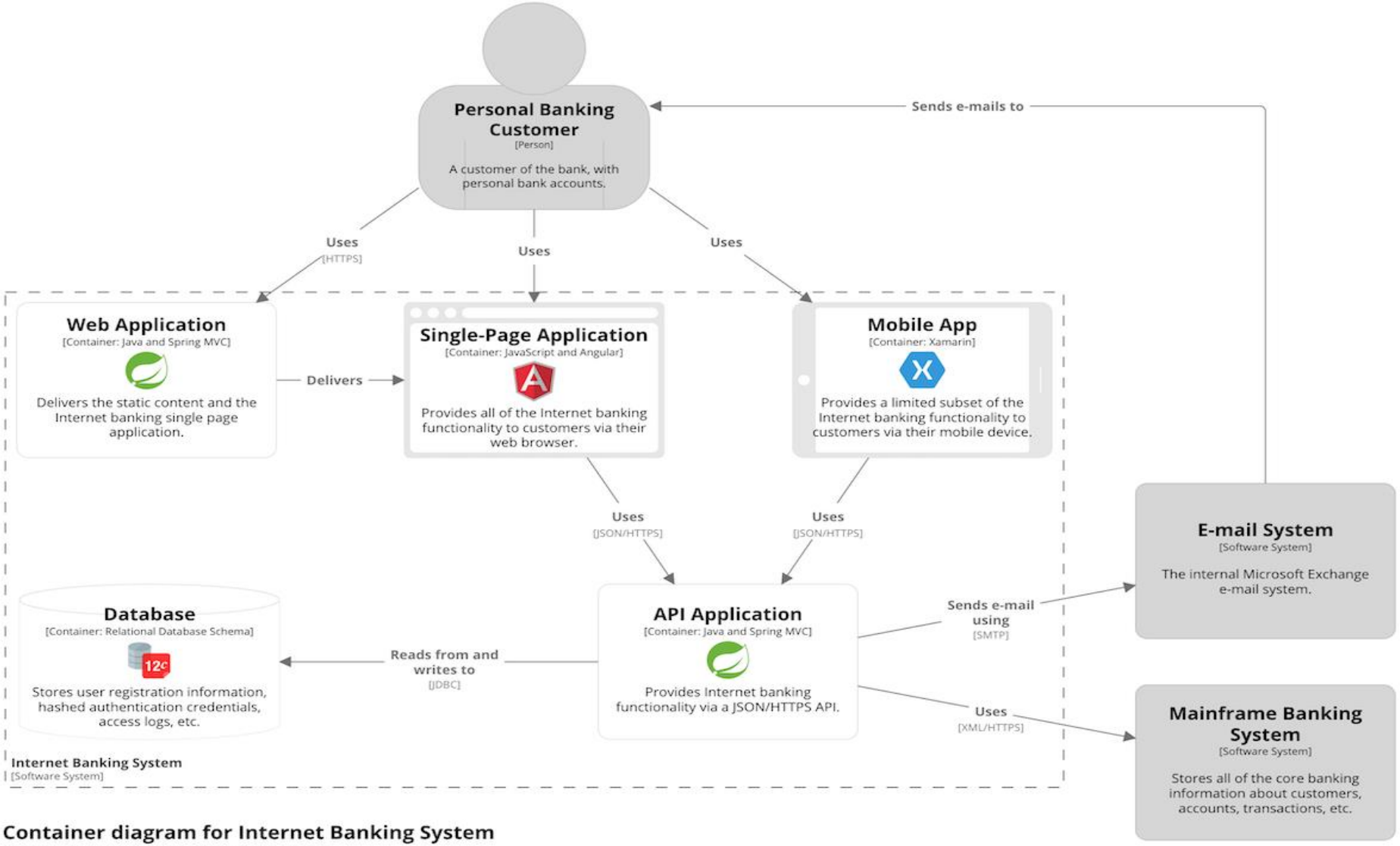
contd..


Client A
HTTP
Firewall
REST Service A
L
Physical Server 1
Physical Server 2
cache
Database
Client B
cache
HTTP
HTTP
Client C
cache
HTTP
Firewall
REST Service B
HTTP
REST Service C
DNS

contd..


Management Tools
OpenEdge Explorer/
OpenEdge Management
RESTman
Java Servlet Container
REST Management Agent
REST Application #1
REST Application #2
REST Application #3
REST Application #4
Web Server
ABL Business Logic
ABL Business Logic
ABL Business Logic
Clients
AppServers

contd..



Container diagram for Internet Banking System

The container diagram for the Internet Banking System.
Workspace last modified: Wed Nov 21 2018 15:02:17 GMT+0000 (Greenwich Mean Time)

REST Anatomy

- A request is formed with following components –
- **Endpoint**: a point marking the **completion of a process** or **stage of a process** especially : a point in a titration at which a definite effect (such as a color change) is observed ; the touchpoints of system–to–system communication are considered **endpoints**, for APIs, an **endpoint** can include a URL of a server or service where server is listening.
  Each **endpoint** is the location from which APIs can access the resources they need to carry out their function.
- **Data**: (also called body or message) contains info user want to send to the server.
- **Method**: User can either request data or modify it, but the server will know the kind of operation the client wants to perform, by implements multiple 'methods' for different types of request, the following are most popular:

contd..

- - **GET**: Get resource from the server.
  - **POST**: Create resource to the server.
  - **PATCH** or **PUT**: Update existing resource on the server.
  - **DELETE**: Delete existing resource from the server
- **Headers**: The additional details provided for communication between client and server (remember, REST is stateless). Some of the common headers are:

  Request:

  - *host*: the IP of client (or from where request originated)
  - *accept-language*: language understandable by the client
  - *user-agent*: data about client, operating system and vendor

  Response:

  - *status*: the status of request or HTTP code.
  - *content-type*: type of resource sent by server.
  - *set-cookie*: sets cookies by server

Resource

- **Resource** – In REST, **resource is data and functionality**;
- Resources are identified by global IDs which are typically URIs, and accessed via a common interface based on HTTP methods.
- The REST server provides access to the resources and REST client can access and modify the resources.
- In RESTful service response is received in different formats like *HTML*, *XML*, *JSON*, *TEXT*, *PDF* and in the *Image* formats.
- REST guidelines specify stateless communication between C–S. Hence, REST always prefers to choose HTTP as it is a stateless protocol.

(Stateless means, every single request from client to server will be considered as a fresh request).

HTTP Methods

- REST based architectures use PUT, GET, POST and DELETE methods.
- PUT – creates a new resource. It must be idempotent, i.e. the request has no side effects.
- GET – defines reading access of the resource. The resource is never changed via a GET request.
- POST – updates an existing resource or creates a new resource.
- DELETE – removes the resources. The operations are idempotent.

REST API Implementations

- There are two major implementations of JAX-RS API.

1. **Jersey**: Jersey is the reference implementation provided by Sun. To use Jersey, configure its servlet in web.xml and add required dependencies.
   a. Jersey is an implementation for the JAX-RS specification.
   b. Jersey provides its own API that extends the JAX-RS toolkit with additional features and utilities.
   c. **Jersey RESTful Web Services** is an open source framework for developing RESTful web services in Java. It provides support for JAX-RS APIs and servers as JAX-RS. It is available from Java 7 and higher.
2. **RESTEasy**: RESTEasy is the JBoss project that provides JAX-RS implementation.

Annotations

1. **@GET –** a method level of annotation, indicating that the following method should respond to the HTTP GET request.
2. **@POST –** a method level of annotation, this annotation indicates that the following method should respond to HTTP POST request only.
3. **@PUT –** a method level of annotation, this annotation indicates that the following method should respond to HTTP PUT request only.
4. **@DELETE** a method level of annotation, indicates that the following method should respond to HTTP DELETE request only.
5. **@Path –** a Class & Method level of annotation, it checks the path next to the base URL

   Base URL : h*ttp://localhost:(port)/<ApplicationName>/<UrlPattern in web.xml>/<path>*

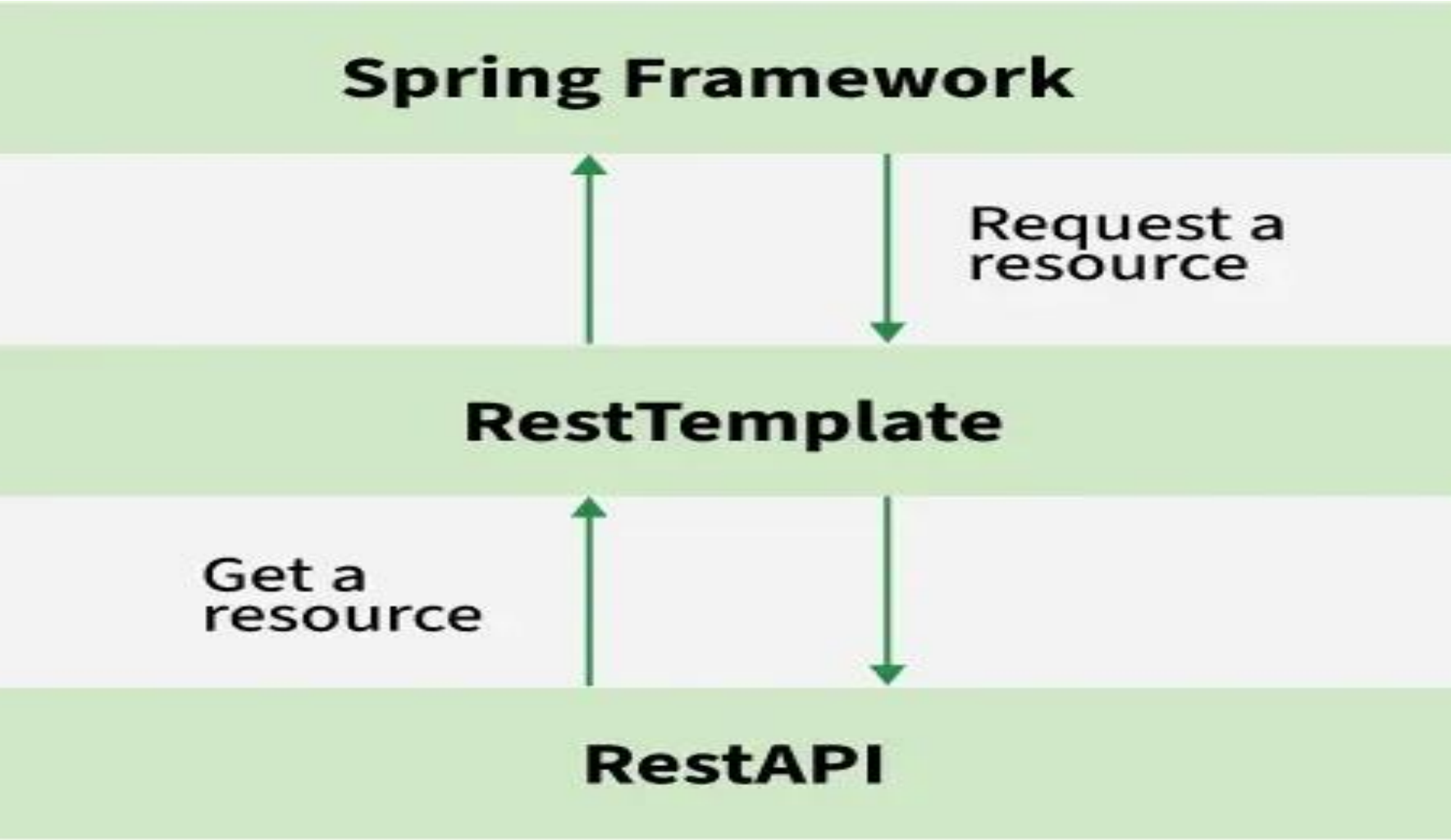
contd..

5. **@Produces** a method or field level annotation, tells which MIME type is delivered by the method annotated with @GET. Meaning when–ever send a HTTP GET request to our RESTful service, it will invokes particular method and produces the output in different formats.
   Note: Use @Produces annotation for GET requests only.
6. **@Consumes** is a class and method level annotation, will define which MIME type is consumed by the particular method, in which format the method can accept the input from the client.
7. Other annotations –
   @PathParam, @QueryParam, @MatrixParam, @FormParam

Class RestTemplate

- Synchronous client to perform HTTP requests, exposing a simple, template method API over underlying HTTP client libraries such as the JDK HttpURLConnection, Apache HttpComponents, and others.
- RestTemplate offers templates for common scenarios by HTTP method, in addition to the
  and execute methods that support less frequent cases.
- RestTemplate is typically used as a shared component. However, its configuration does not support concurrent modification, and as such its configuration is typically prepared on startup.
- If necessary, multiple, differently configured RestTemplate generalized exchange instances can be created on startup. Such instances may use the same underlying ClientHttpRequestFactory if they need to share HTTP client resources.

Diagrammatic Representation

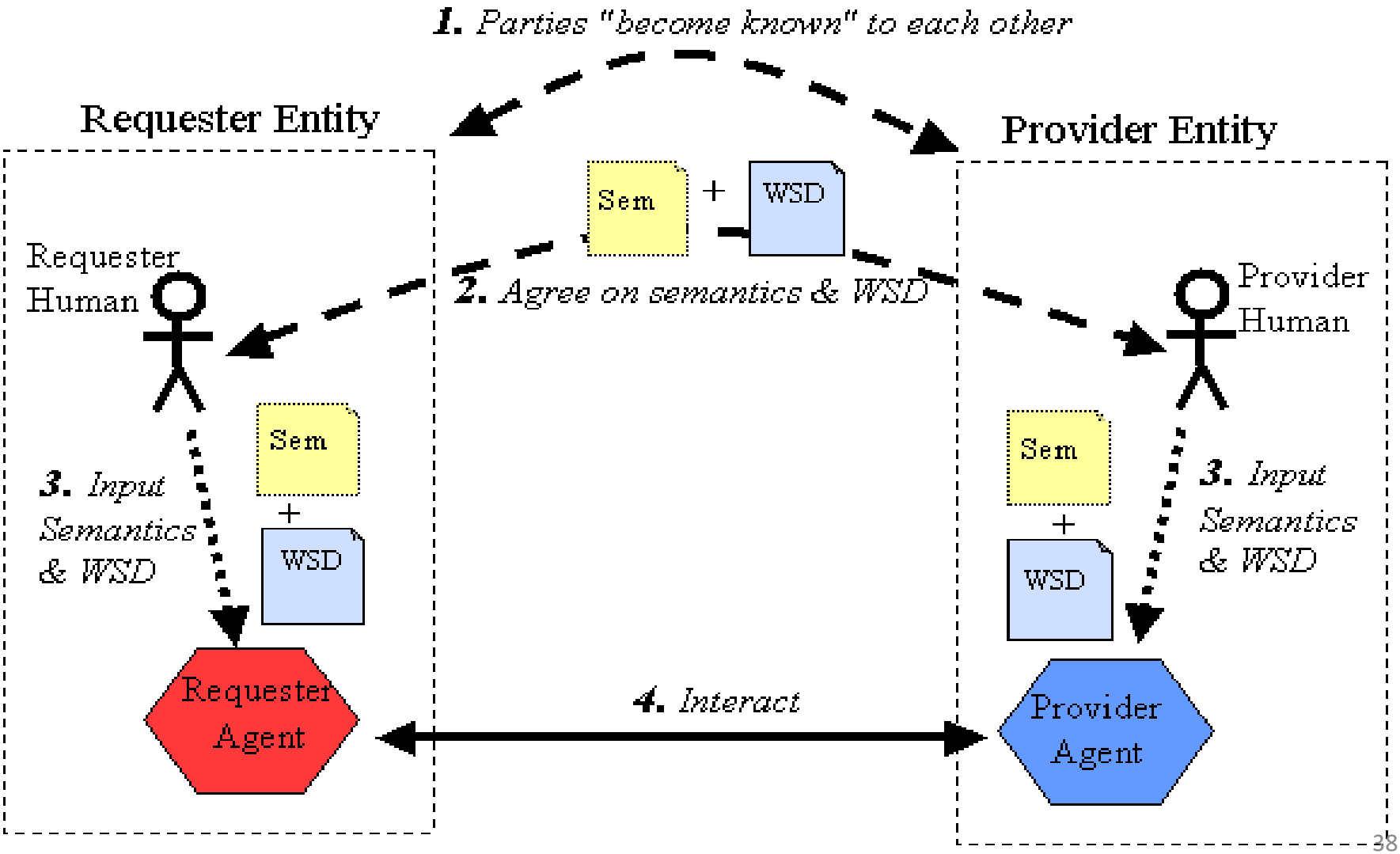


contd..

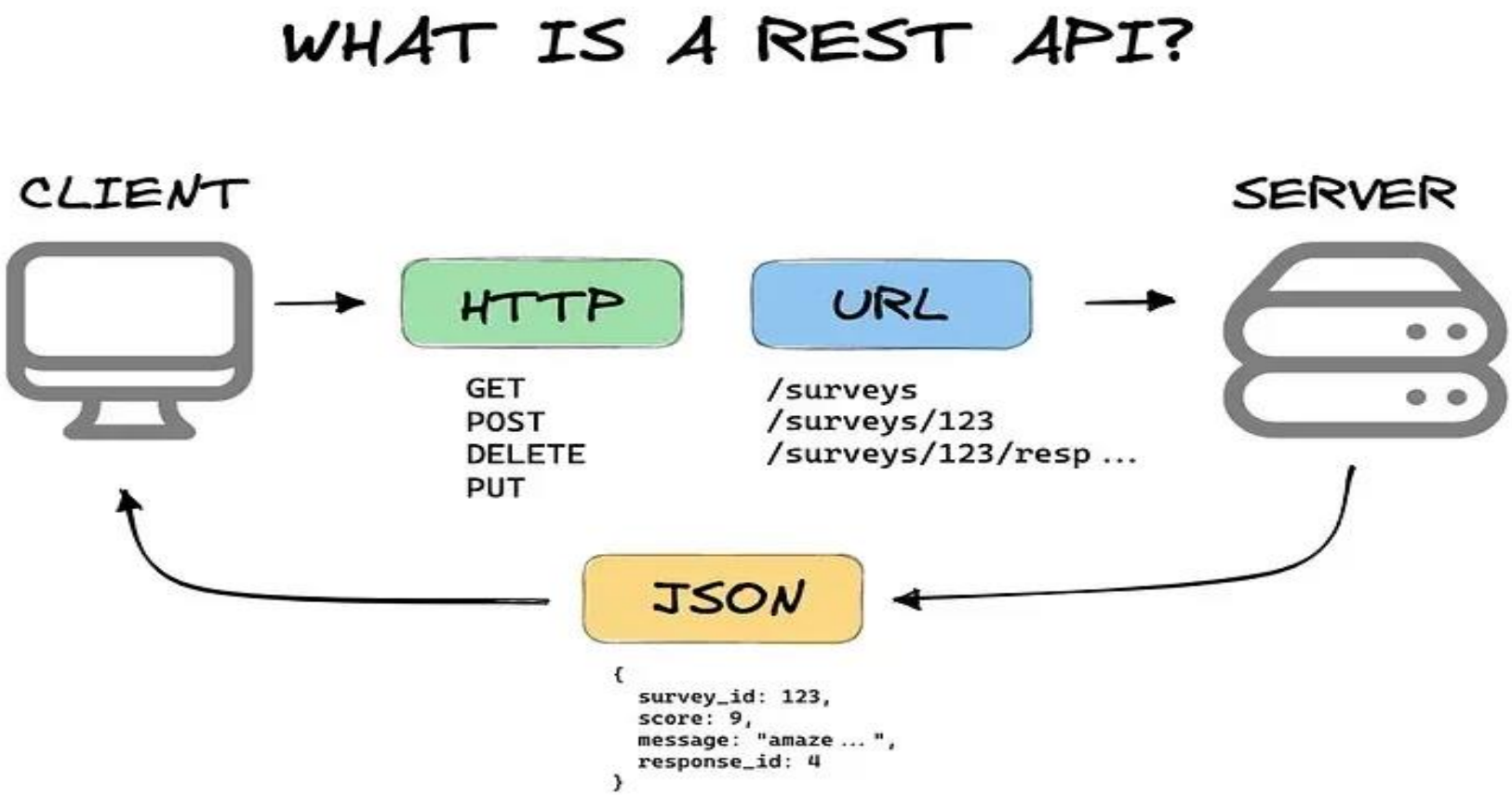
- **@LoadBalanced** annotation is used specifically with RestTemplate in to **intercept** HTTP requests and **enable client-side load balancing**, often in conjunction with service discovery tools like Netflix Eureka.
- Why it is uniquely paired with RestTemplate and how it works -
  - ✓ **Intercepting Requests:** The annotation tells Spring to intercept the RestTemplate bean creation and apply LoadBalancerInterceptor.
  - ✓ **Service Name Resolution:** Without this annotation, RestTemplate treats URLs as literal addresses (e.g., http://localhost:8080). With @LoadBalanced, it understands logical service names (e.g., http://user-service/api/users) and resolves them to actual IP addresses/ports of available instances via a service registry.
  - ✓ **Client-Side Load Balancing:** When multiple instances of a service are registered in Eureka, the load balancer client (like Ribbon or Spring Cloud LoadBalancer) chooses one in a round-robin fashion.
  - ✓ **Legacy Context:** While modern alternatives like WebClient or Feign are now preferred, @LoadBalanced was the standard mechanism to make RestTemplate work in a service-to-service microservices architecture.

public interface WebClient

- Non-blocking, reactive client to perform HTTP requests, exposing a fluent, reactive API over underlying HTTP client libraries such as Reactor Netty.
- has a more modern API and supports sync, async, and streaming scenarios.
- Use static factory methods create() or create(String), or builder() to prepare an instance.


1. Parties "become known" to each other
Requester Entity
Provider Entity
Sem
+
WSD
2. Agree on semantics & WSD
Requester Human
Provider Human
Sem
+
WSD
3. Input Semantics & WSD
Sem
+
WSD
3. Input Semantics & WSD
Requester Agent
4. Interact
Provider Agent

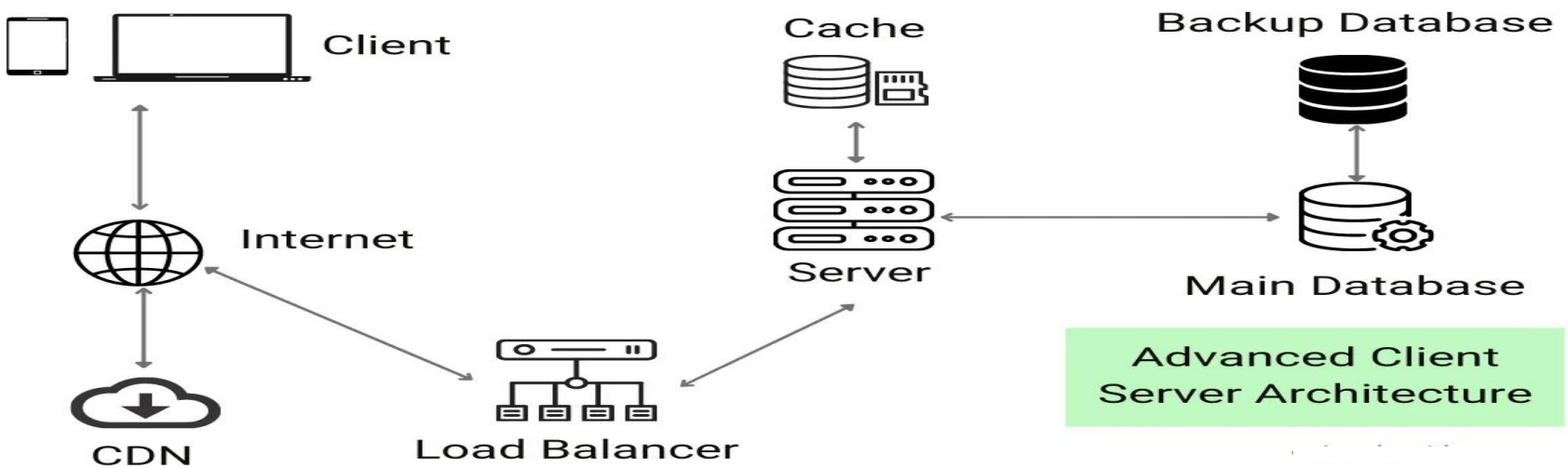
- A web service architecture typically consists of three main components: a Service Provider (hosts the service), a Service Requestor (client using the service), and a Service Registry (UDDI directory for discovery). They interact via Publish, Find, and Bind operations over a network using XML-based protocols like SOAP/WSDL.
- Key Components of Web Service Architecture
- Service Provider (Server): Implements the web service, creates the WSDL interface, and makes it available to the public or private network.
- Service Requestor (Client): The application or user that needs a functionality, which contacts the registry to find the service and then binds to the provider.
- Service Registry (Broker): A directory that stores service descriptions (UDDI) to help requestors find providers.
- Communication Protocols:
- SOAP: Messaging protocol for sending XML requests.
- WSDL: Describes the service interface, location, and methods.
- UDDI: Enables discovery of services.


WHAT IS A REST API?
CLIENT
SERVER
HTTP
URL
GET
POST
DELETE
PUT
/surveys
/surveys/123
/surveys/123/resp ...
JSON
{
survey_id: 123,
score: 9,
message: "amaze ... ",
response_id: 4
}

- REST Key Components
- Resources: Key entities exposed by the API, each identified by a unique URI.
- HTTP Methods: Used to perform CRUD operations on resources (GET, POST, PUT, DELETE).
- Representations: Data format representing resource state (e.g., JSON, XML).
- Hypermedia Links: Embedded links in responses to enable dynamic navigation.
- Status Codes: Indicate the status of a request (e.g., 200 OK, 404 Not Found).

- 2. Uniform Interface
- 2.1. Identification of Resources:
- Resources are uniquely identified by URIs.
- Example: /api/users, /api/products
- 2.2. Manipulation of Resources through Representations:
- Resources have uniform representations in server responses.
- **2.3. Self-Descriptive Messages:**
- Resource representations carry information to describe processing.
- **2.4. Hypermedia as the Engine of Application State (HATEOAS):**
- Clients navigate resources and interactions using hyperlinks.
- The blog post resource includes hypermedia links to related resources:
- "author": Links to the author resource, allowing clients to retrieve author details or posts written by the author.
- "comments": Links to the comments resource for the specific post, enabling clients to retrieve comments associated with the post.
- Each hyperlink includes a rel attribute indicating the relationship type and a href attribute providing the URI of the related resource.
- Clients can navigate the API dynamically by following these hyperlinks, discovering and accessing related resources as needed.

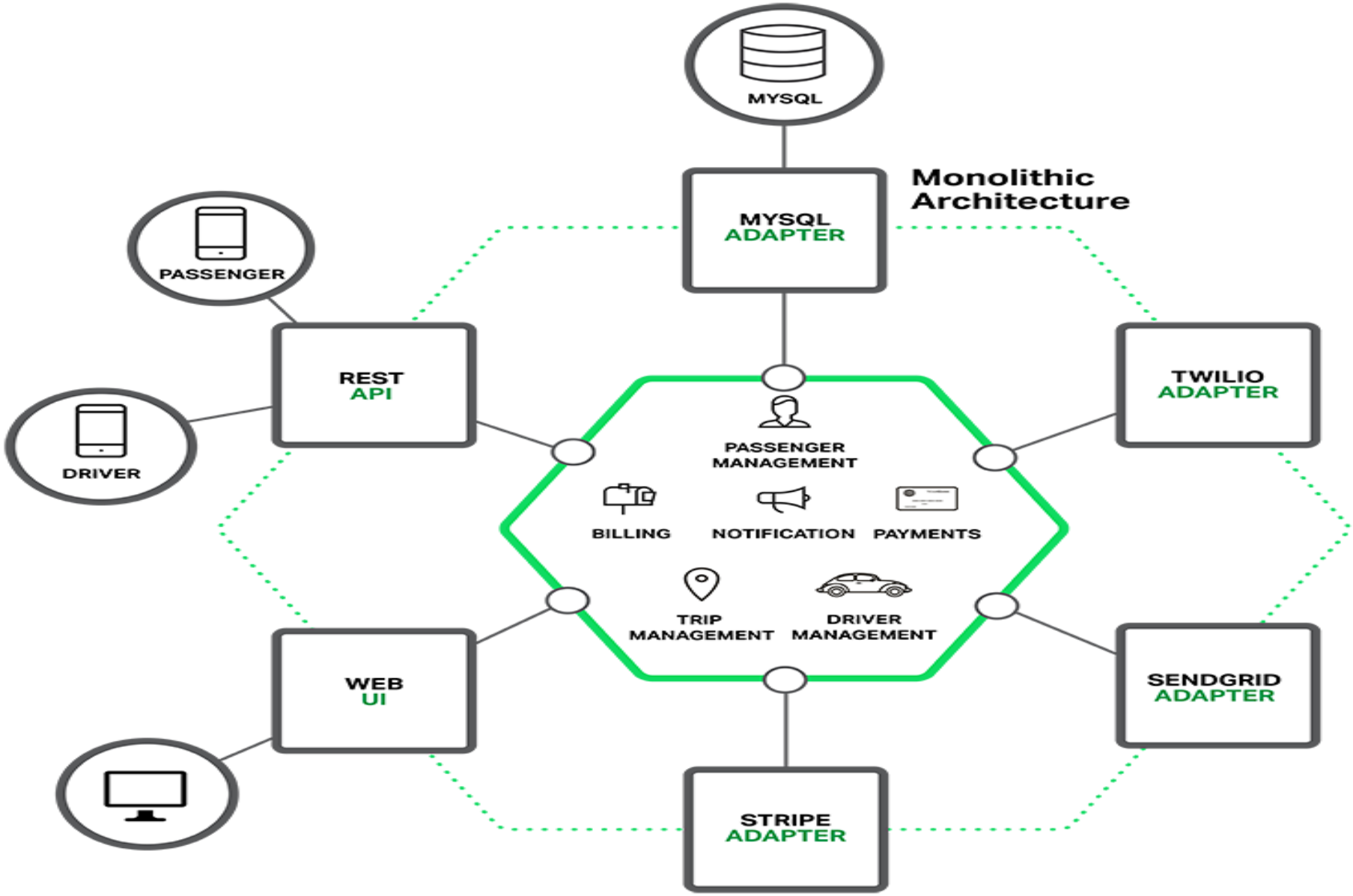
Client-Server Architecture



- Separation of concerns between client and server components where in Clients request resources, and server provide responses. Promotes scalability and independent evolution of client and server.
- E.g. scenario where client-side web application communicates with a server-side REST API to retrieve and manipulate user data.
- **Client-side –** web application built using HTML, CSS & JavaScript running on user's web browser.
- **Server-side –** backend server implemented using frameworks like Express.js, Django exposing RESTful endpoints to handle clien-req.

- 4. Statelessness:
- Each request from client to server must contain all necessary information.
- Servers do not maintain client state between requests.
- Enhances scalability and reliability.
- 5. Cacheability
- Responses from the server can be labeled as cacheable or non-cacheable using appropriate HTTP headers.
- Caching improves performance and reduces server load by serving cached responses for subsequent identical requests
- 6. Layered System
- Clients interact with a layered architecture of servers, providing an abstraction of services.
- Intermediate servers can be used for load balancing, caching, security, etc., without affecting the client-server interaction.

Monolith Architecture



contd..

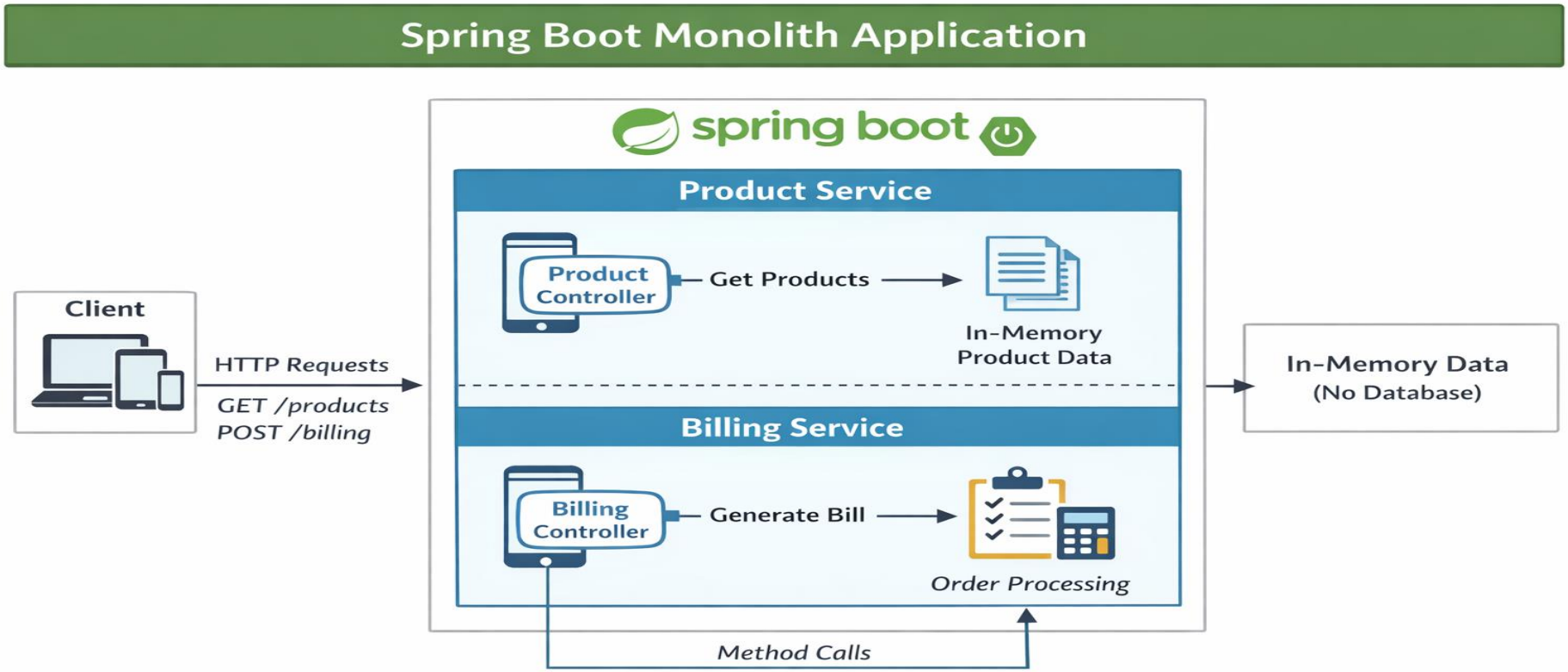
- Core of the application is **business logic -** implemented by modules defining **services, domain objects**, and **events**. Packaging and deployment is **monolith**, though its a logically modular architecture.
- **Adapters** surround the core interfacing with the external world. E.g. database access components, messaging and web components.
- Java applications are packaged as WAR files and deployed on application servers such as Tomcat or Jetty. Similarly, Rails and Node.js applications are packaged as a directory hierarchy.
- Applications developed in this style are easy to develop, test and deploy as IDEs, tools, UI – Selenium, are focused on building and testing single application. Deploying a monolith involves to copy the **packaged application to a server**.
- The application can be scaled by running multiple copies behind a load balancer. In the early stages of the project it works well.

Issues with Monolith

- Applications grow over time and eventually becoming huge. At each sprint, the development team implements a **few more issues**, which, of course, means **adding many lines of code**.
- The application is complex and too large for any single developer to fully understand, hence to fix bugs, implement new features is time consuming. Result is a**gile development** and **delivery** will flounder which tends to be a downwards spiral .
- Larger applications, have long start-up time. A survey reported the **start-up time** as **12 minutes**.
- **Monolithic** applications obstruct **continuous deployment,** difficult to **scale** as different modules have **conflicting** resource needs.
- There are issues of **reliability** as all modules are running within the same process, a bug in any module, such as a memory leak, can potentially bring down the entire process.
- Monolith applications are extremely difficult to adopt new frameworks and languages.

contd..

- This is a **Monolith Architecture**:
- One application
- One deployment
- Services communicate via **method calls (not REST)**



contd..

- Control flow

Client

↓

Controller Layer

↓

Service Layer (User ↔ Billing ↔ Logistics)

↓

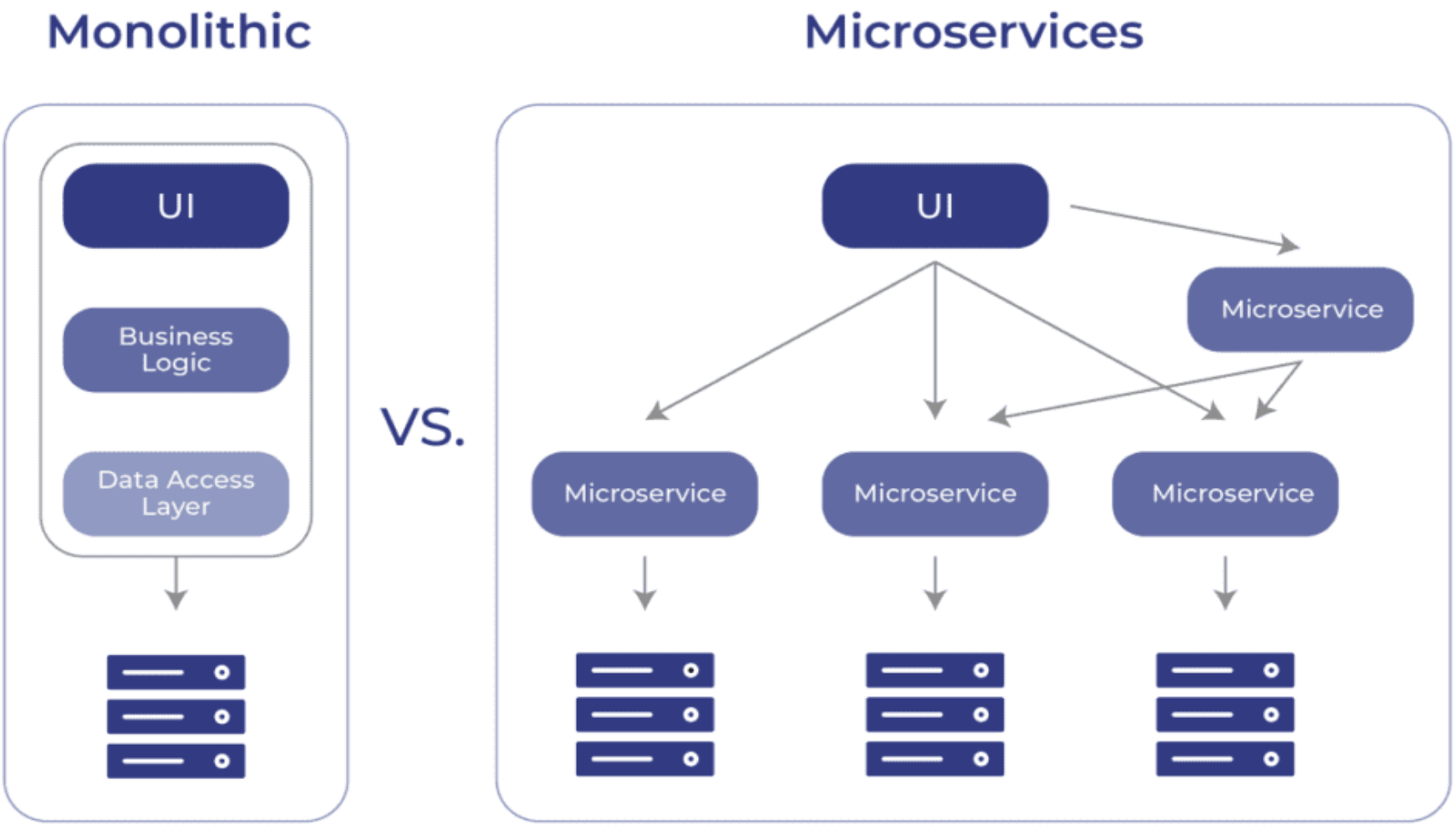
In-Memory Data (HashMap)

No DB
Services interact via **method calls (tight coupling)**
Single deployable unit

MicroServices

- Microservices architecture is a variant of service-oriented architecture, an architectural paradigm for building autonomous, flexible, extensible, independently deployable, and fine-grained services.
- Microservice – **service-based development methodology** wherein **big application** is divided into small, autonomous, independent, interconnected **service units** each serving a **single business need** and are **loosely coupled** making it easier to **update**, **maintain,** and **scale** the service unit independent of each other.
- Microservices can be written in different programming languages and use different technologies, as long as they can communicate with each other through a common API.
- Microservice is the process of implementing Service-oriented Architecture (SOA) design pattern.
- **Sam Newman** defined microservice as Small autonomous services that work together.

Diagrammatic Representation



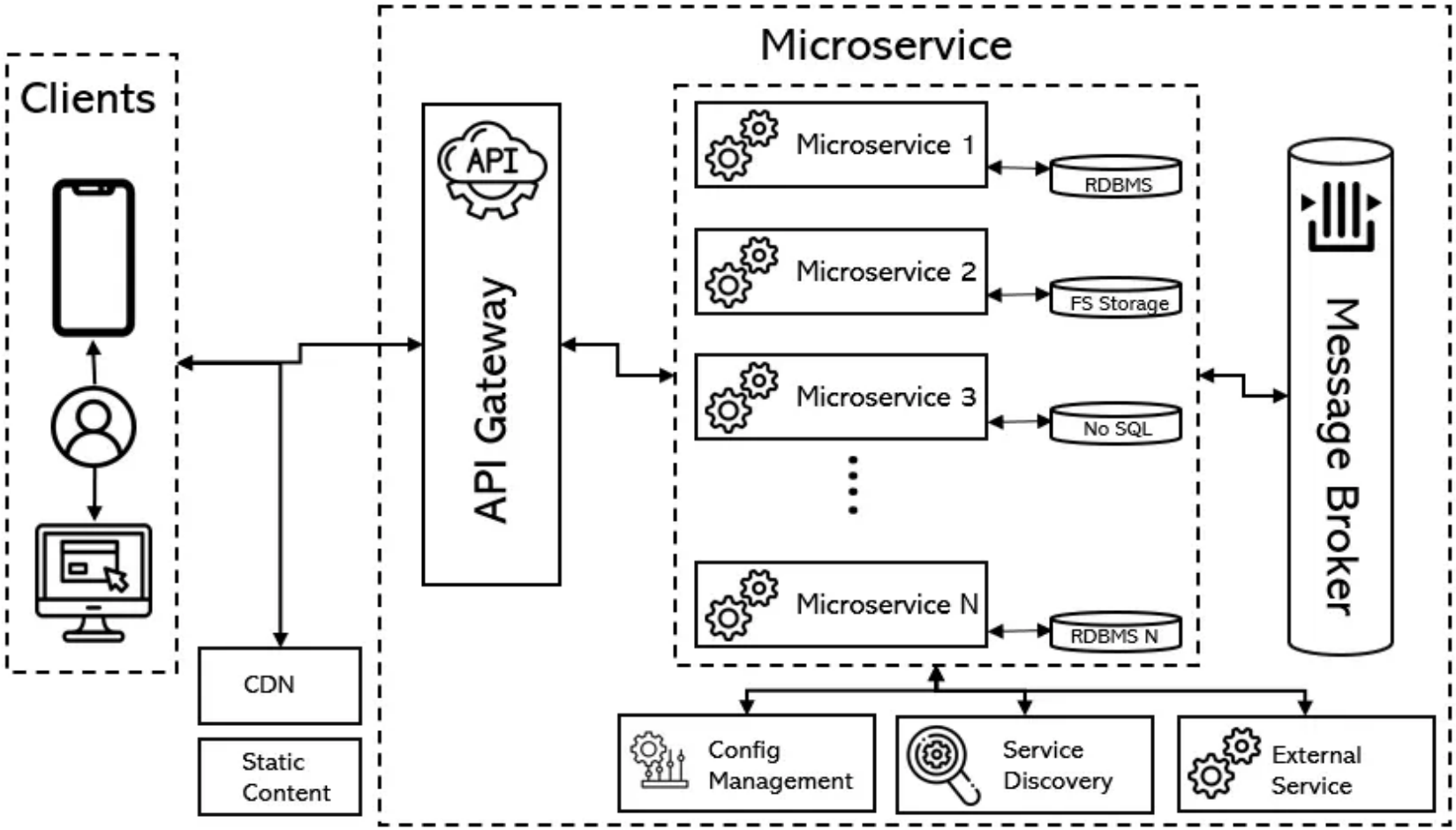
Key Micro Services Components

1. **Core Services**: Each service is a self-contained unit of functionality that can be **developed**, **tested,** and **deployed** independently.
2. **Service registry**: Service Registry is a database of all the services in the system, along with their locations and capabilities. A guide to discover the communication route between services.
3. **API Gateway:** An API gateway is a single entry point for all incoming requests to services. It acts as a reverse proxy, routing requests to the appropriate service and handling tasks such as authentication and rate limiting.
4. **Message bus:** A message bus is a messaging system that allows services to communicate asynchronously with each other. This can be done through protocols like HTTP, RabbitMQ, or Kafka.
5. **Monitoring and logging:** Monitoring and logging are necessary to track the health of the services and troubleshoot problems.
6. **Clients** : Different users from various devices send requests.

contd..

7. **Static Content** : Houses all the content of the system.
8. **Service discovery and load balancing :** This component is responsible for discovering service instances and directing traffic to the appropriate service instances based on load and availability.
9. **Management** : Balances services on nodes and identifies failures.
10. **Content Delivery Networks** : Distributed network of proxy servers and their data centers.
11. **Remote Service** : Enables the remote access information that resides on a network of IT devices.
12. **Continuous integration and continuous deployment (CI/CD) :** To make the development and deployment process of microservices as smooth as possible, it is recommended to use a tool such as Jenkins, TravisCI, or CircleCI to automate the process of building, testing, and deploying microservices.

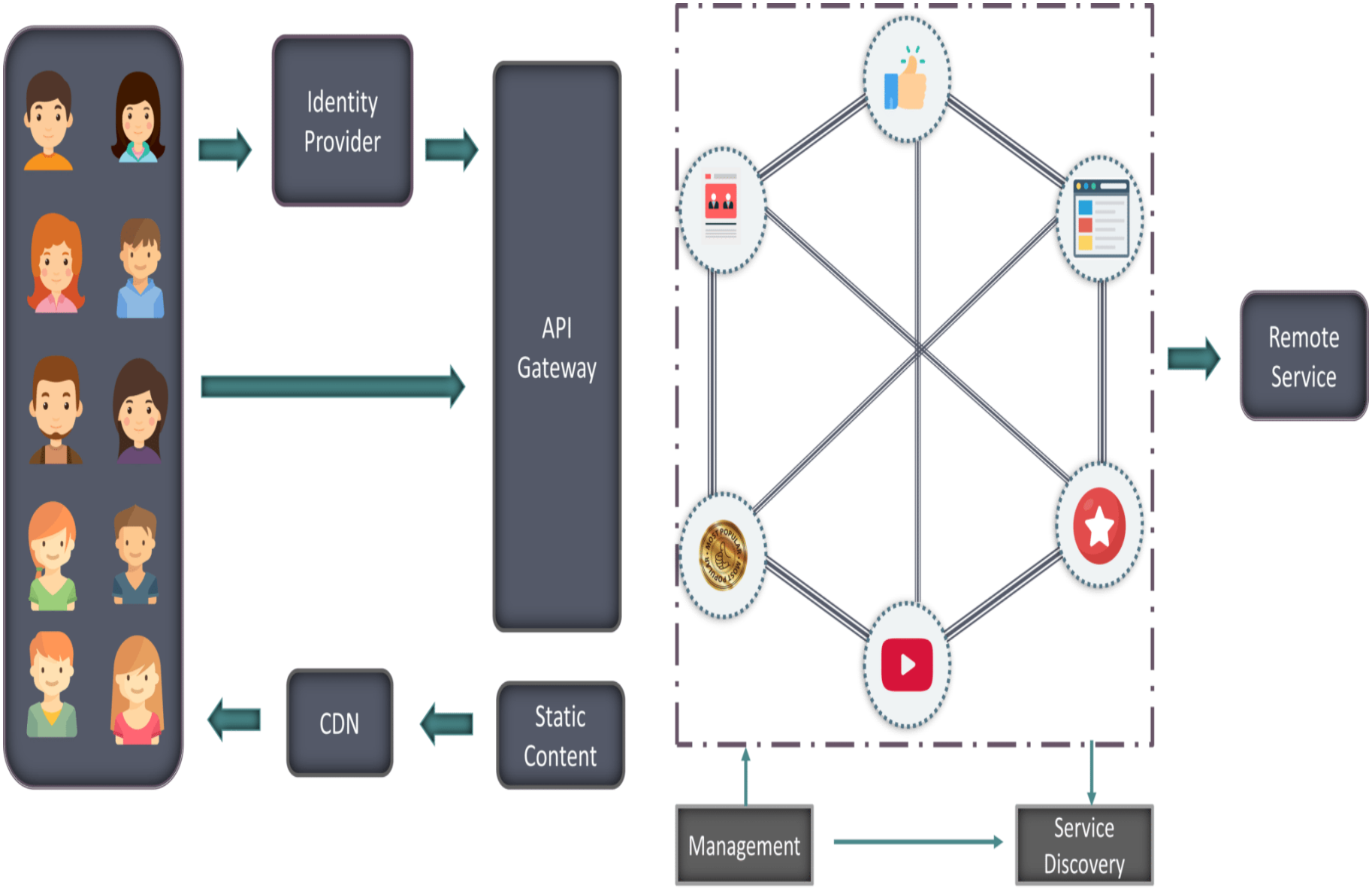
Diagrammatic Representation



Microservices Architecture

Sumit Kumar Sharma

Diagrammatic Representation

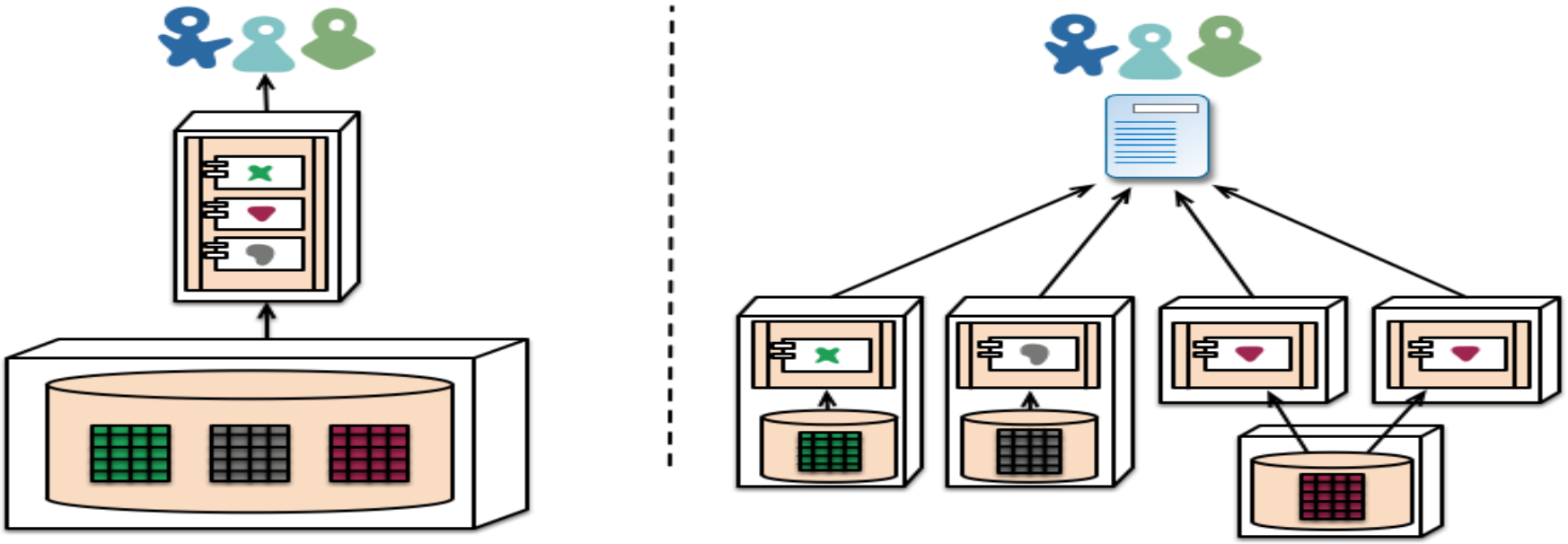


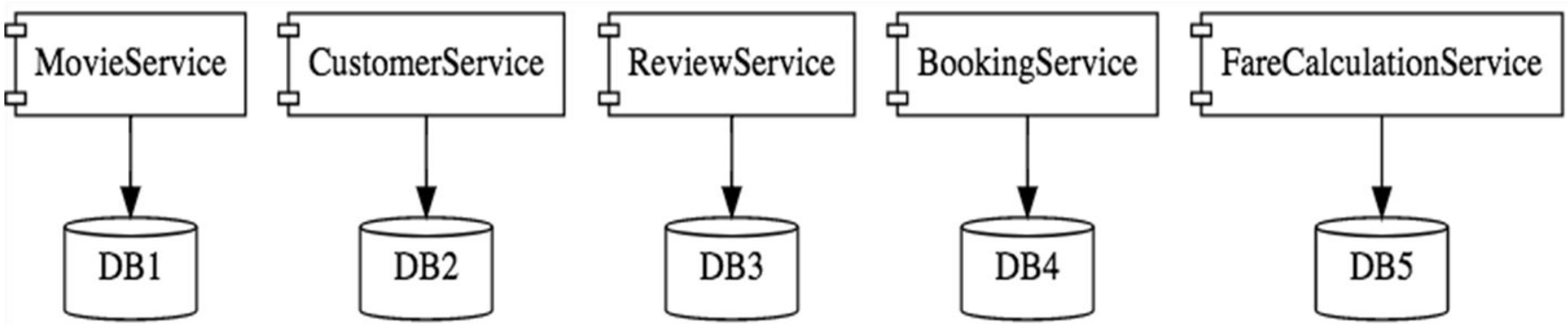
Microservice Architecture

- Microservice Architectures evolved as a solution to the **scalability** and **innovation** challenges with Monolith architectures.
  - ✓ **MicroServices** is a form of service-oriented architecture style, wherein applications are built as a collection of different smaller services rather than one whole app
  - ✓ The application is functionally decomposed into a collection of **loosely coupled services.**
  - ✓ Each service has a narrow, focused set of responsibilities, and is, in some cases, quite small.
  - ✓ It improves modularity, thus making the application easier to develop, test & deploy.
  - ✓ The development process is more efficient by parallelizing small teams to work on different services.

contd..

- Microservice Architectures involve a number of small, well designed, components interacting with messages.



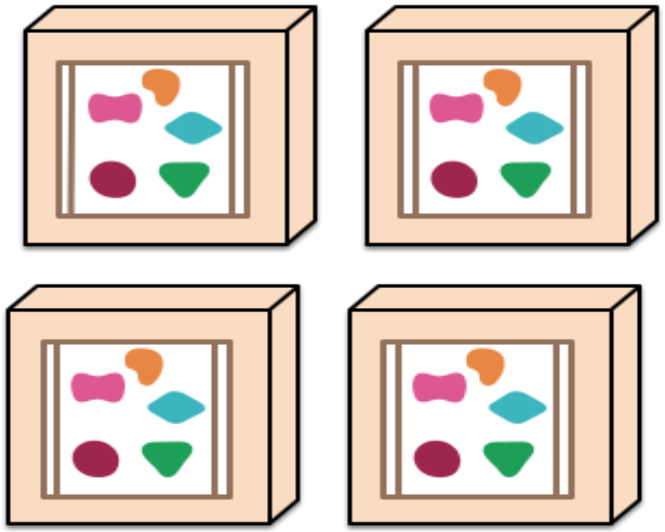


contd..

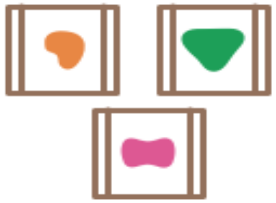
- Deployment and Scaling

A monolithic application puts all its functionality into a single process...

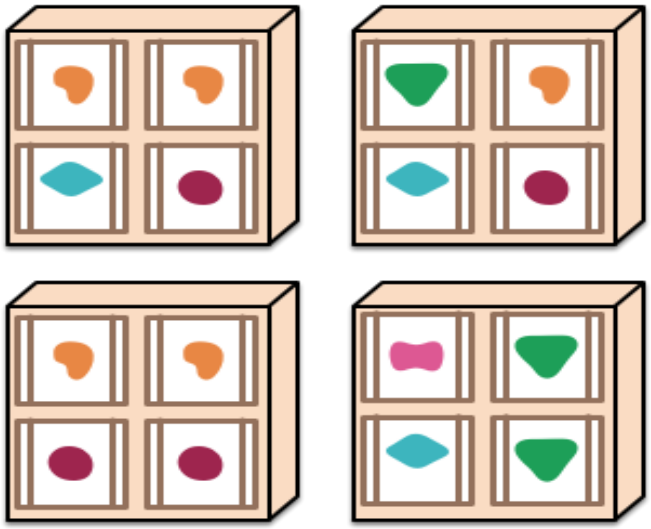
... and scales by replicating the monolith on multiple servers



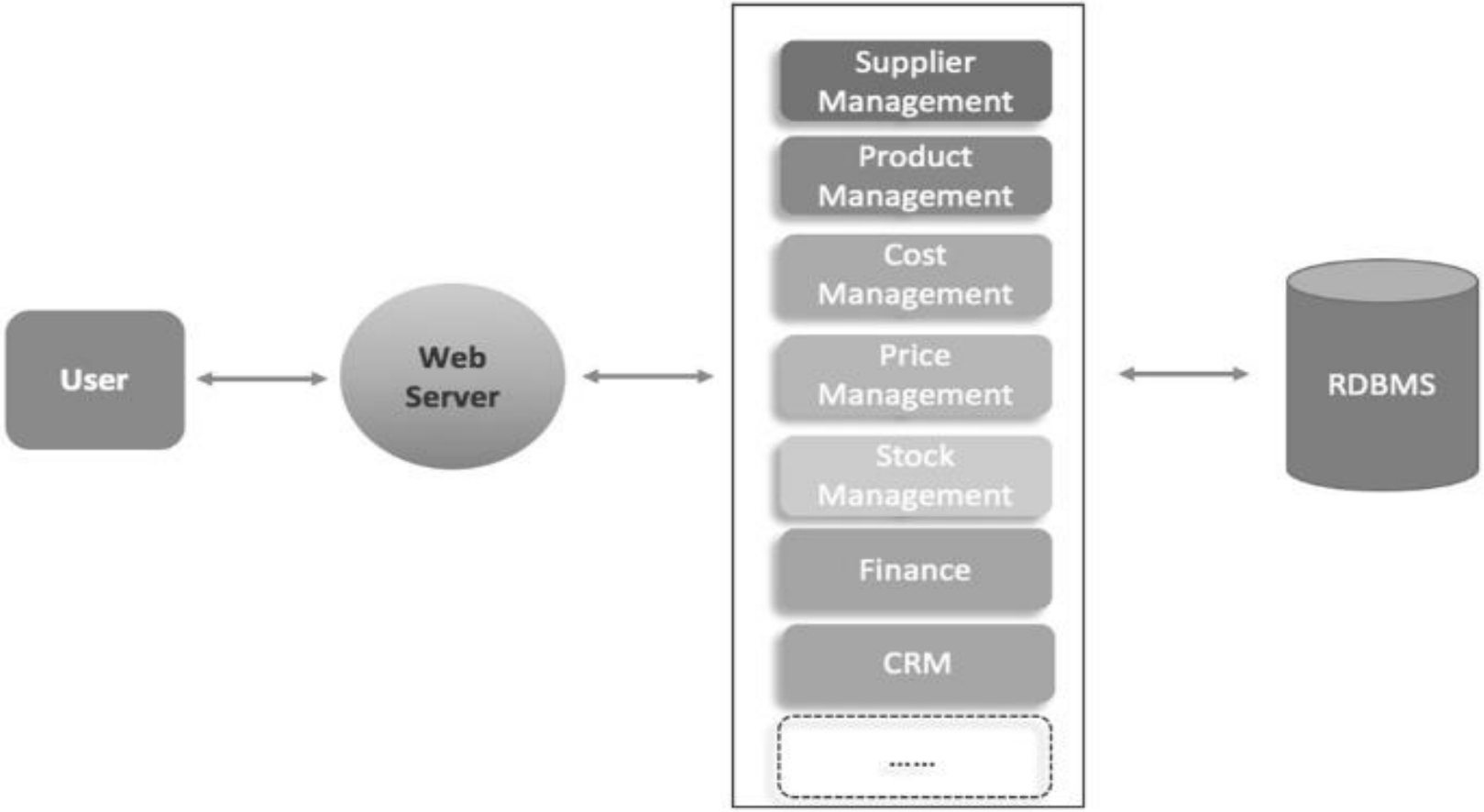
A microservices architecture puts each element of functionality into a separate service...

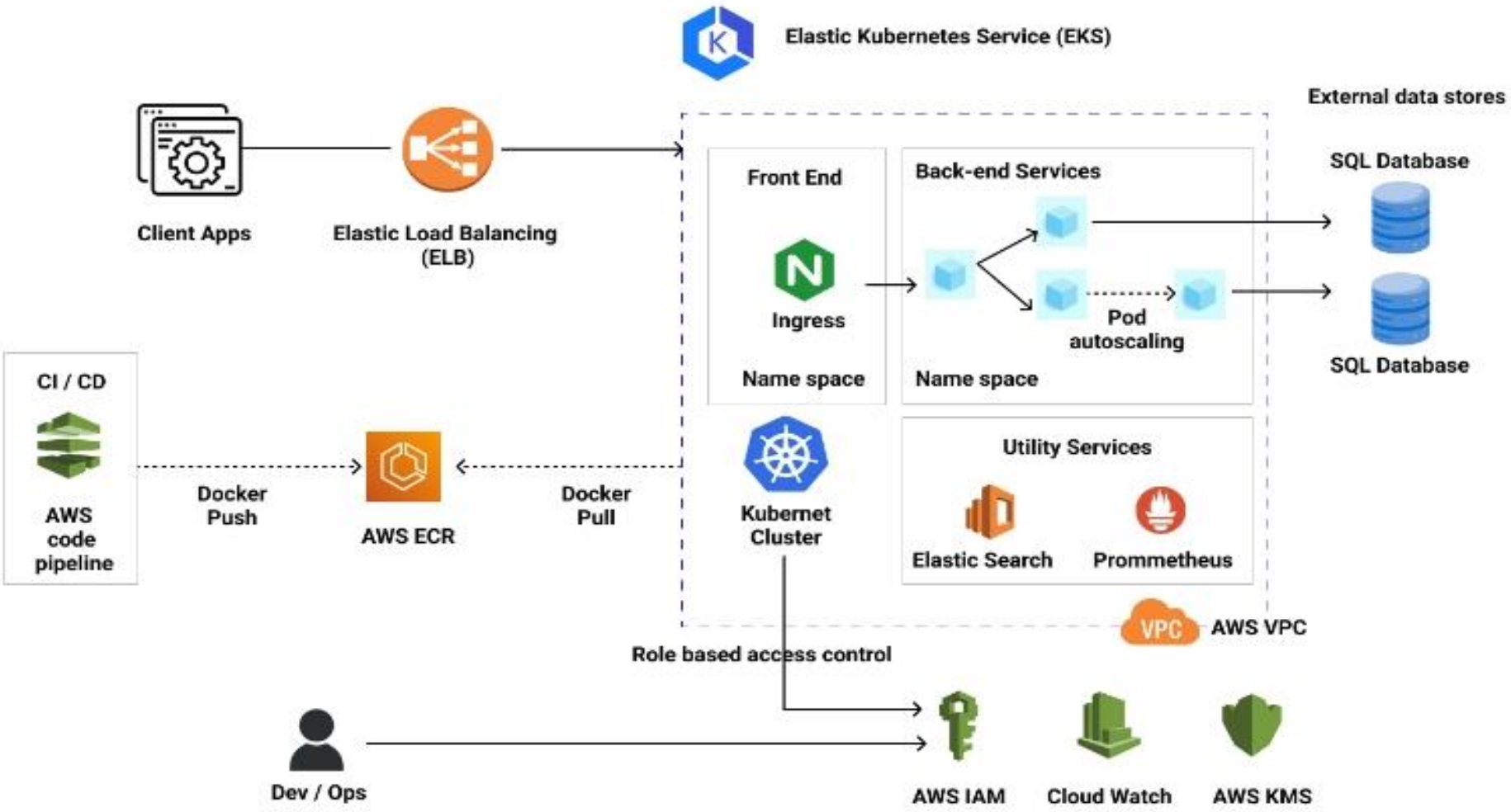


... and scales by distributing these services across servers, replicating as needed.



A generic monolith system




Elastic Kubernetes Service (EKS)
External data stores
Client Apps
Elastic Load Balancing (ELB)
Front End
Ingress
Name space
Back-end Services
Pod autoscaling
Name space
SQL Database
SQL Database
CI / CD
AWS code pipeline
Docker Push
AWS ECR
Docker Pull
Kubernet Cluster
Utility Services
Elastic Search
Prommetheus
VPC
AWS VPC
Role based access control
Dev / Ops
AWS IAM
Cloud Watch
AWS KMS

Approach to develop Microservice application

- **Microservices architecture** is best suited for large, and complex applications that need to handle a high volume of traffic and need to be scaled horizontally.
- **Independent**: Each microservice should be independently deployable, loosely coupled with one another such that changes in one will not affect the other, capable of delivering one specific business goal.
- **Small-size**: A service should not perform more than one business task, so it will be small in size and easy to maintain, committed to delivering only one business property.
- **Autonomous**: Each microservice is an autonomous business unit of the entire application. So application becomes loosely coupled.
- **Technology heterogeneity**: By implementing a heterogeneous system, max security, speed and a scalability can be achieved.

contd..

- **Resilience**: Resilience is a property of isolating a software unit. Hence if one unit fails it does not impact the entire business.
- **Ease of deployment**: The app is sub-divided into small units, each can be deployed in any environment very easily with less time complexity

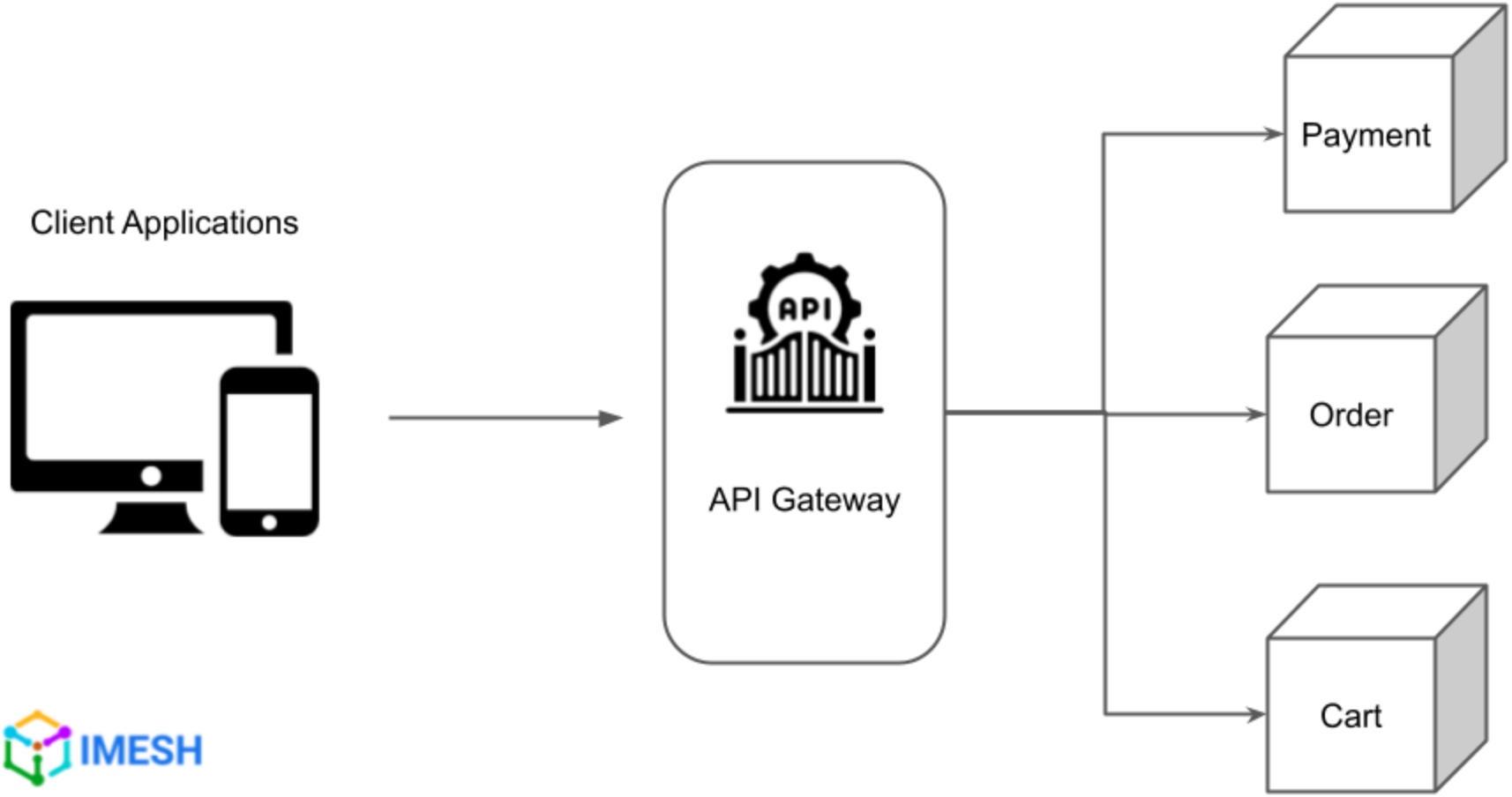
Microservices types

- There are two types of microservices :
  - ✓ Stateless microservices.
  - ✓ Stateful microservices.

API gateways

- **API Gateway** is a **centralized entry point** that manages client requests and directs them to the appropriate backend services.
- Objective is to simplify clients-to-service(s) communication while enforcing security and performance policies. API gateway provides an API for each client.
- API gateways encapsulate the complexity of the underlying system and let clients talk to it instead of calling specific services.
- They also perform security checks (authentication and authorization) before the traffic reaches the service.
  - ✓ Handles request routing, authentication, and rate limiting.
  - ✓ Acts as a reverse proxy, hiding internal service complexity.
  - ✓ Provides a unified and consistent API interface for clients.
  - ✓ ***E.g.*** *in e-commerce system, a single API Gateway can route requests for product details, user orders, and payment processing to separate microservices, with checking user authentication and limiting request rates.*

contd..



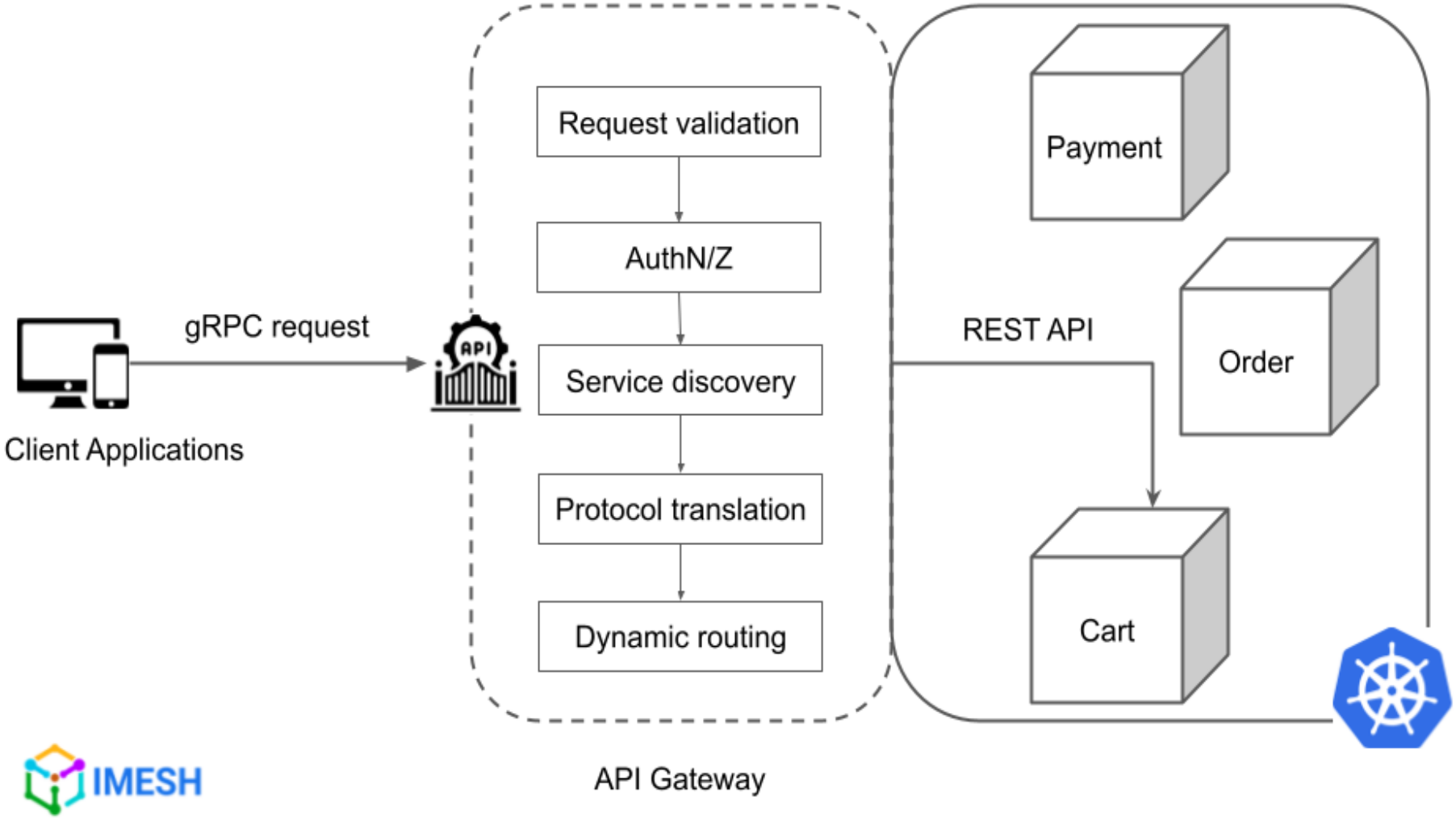
Challenges with direct C2MS communication

- **API gateway traffic flow –** API gateways can read and understand the HTTP messages so can apply filters or take actions on the traffic. Its a L7 proxy that abstracts the traffic management out of front-end microservices, which are usually requested by clients.
- **The problem of service discovery and traffic routing –** the clients have to know the specific endpoints of service instances. Keeping track of endpoints adds complexity for clients because of the dynamic (de)scaling of services. Also, scaling becomes an issue as it will require configuration changes on the client side. Routing traffic based on attributes like geography (geo-routing), is hard to configure when clients invoke services directly.
- **Security concerns –** exposing service endpoints for clients causes security concerns. Increases the attack surface for intruders and leaves backend services prone to threats, e,g. packet sniffing, man-in-the-middle attack, etc. Also it puts the burden of authenticating and authorizing API calls on services, rather than focus on business logic.

contd..

- **Diverse protocols affecting interoperability –** flexibility of microservices architecture lets developers build services using diverse language – Python, Java, Go and implement those services in different API types, such as REST, gRPC, and others. It then becomes a requirement for the client to understand and communicate using different protocols. This adds additional complexity as the client application will require more code and logic.
- **Latency caused by round trips –** Some attributes on client page e.g. product pricing, quantity, and reviews will be deployed as different services in the backend. If the client is invoking services directly, it will have to make separate requests for each service (product price, reviews, quantity, etc.) to retrieve the required information, since there is no mechanism to cache a response from the upstream services. These calls add to the overhead of establishing multiple connections, and the round trips caused by these network requests add to latency and suboptimal user experience.

Diagrammatic Representation



Config Server

- **Config Server** in Spring Boot (specifically Spring Cloud Config) is a **centralized service** used to manage external configuration properties for all microservices across different environments (e.g., development, staging, production).
- In a microservices architecture, instead of hardcoding property files e.g. application.properties inside each individual service, those properties are stored in a central location managed by the Config Server.
- **Centralized Management**: Manage configuration for hundreds of microservices in one place.
- **Environment Specificity**: Use Spring Profiles to serve different values for the same key depending on whether the service is in dev, test, or prod.
- **Dynamic Updates**: Update configuration at runtime without needing to rebuild or restart your microservices.
- **Security**: Support for Encryption and Decryption of sensitive data (like database passwords) before they are sent to the client.

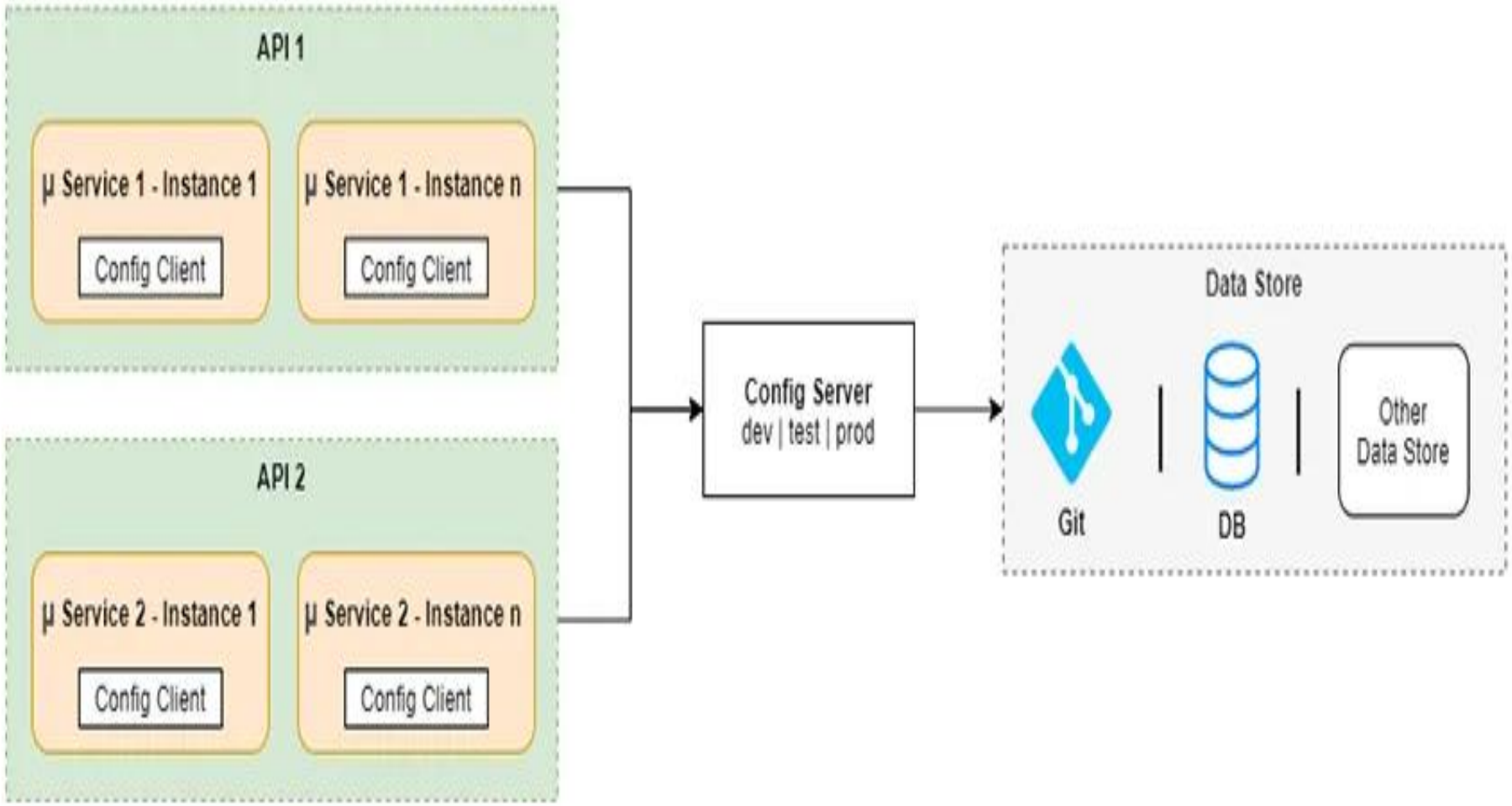
contd..

- **Supported Backends –** The Config Server can retrieve properties from various storage systems:
  - ✓ **Git**: The default and most popular implementation, allowing for versioning and auditing of configuration changes.
  - ✓ **Vault**: For high-security storage of secrets using HashiCorp Vault.
  - ✓ **JDBC**: Storing properties in a relational database.
  - ✓ **Native File System**: Reading files from the local classpath or disk (commonly used for testing).
- **How to Implement It**
  - ✓ **Dependencies**: Add the spring-cloud-config-server dependency to the Spring Boot project.
  - ✓ **Enablement**: Add the @EnableConfigServer annotation to main Spring Boot Application class.
  - ✓ **Client Connection**: Microservices (clients) connect to the server by adding the spring-cloud-starter-config dependency and specifying the server's URI in their configuration.

contd..

- **Why is it Necessary?**
  - ✓ Without a Config Server, changing a single database password would require you to update, rebuild, and redeploy every single microservice that connects to that database.
  - ✓ With a Config Server, you update the property in the central repository, and the services can refresh their state automatically.

Diagrammatic Representation

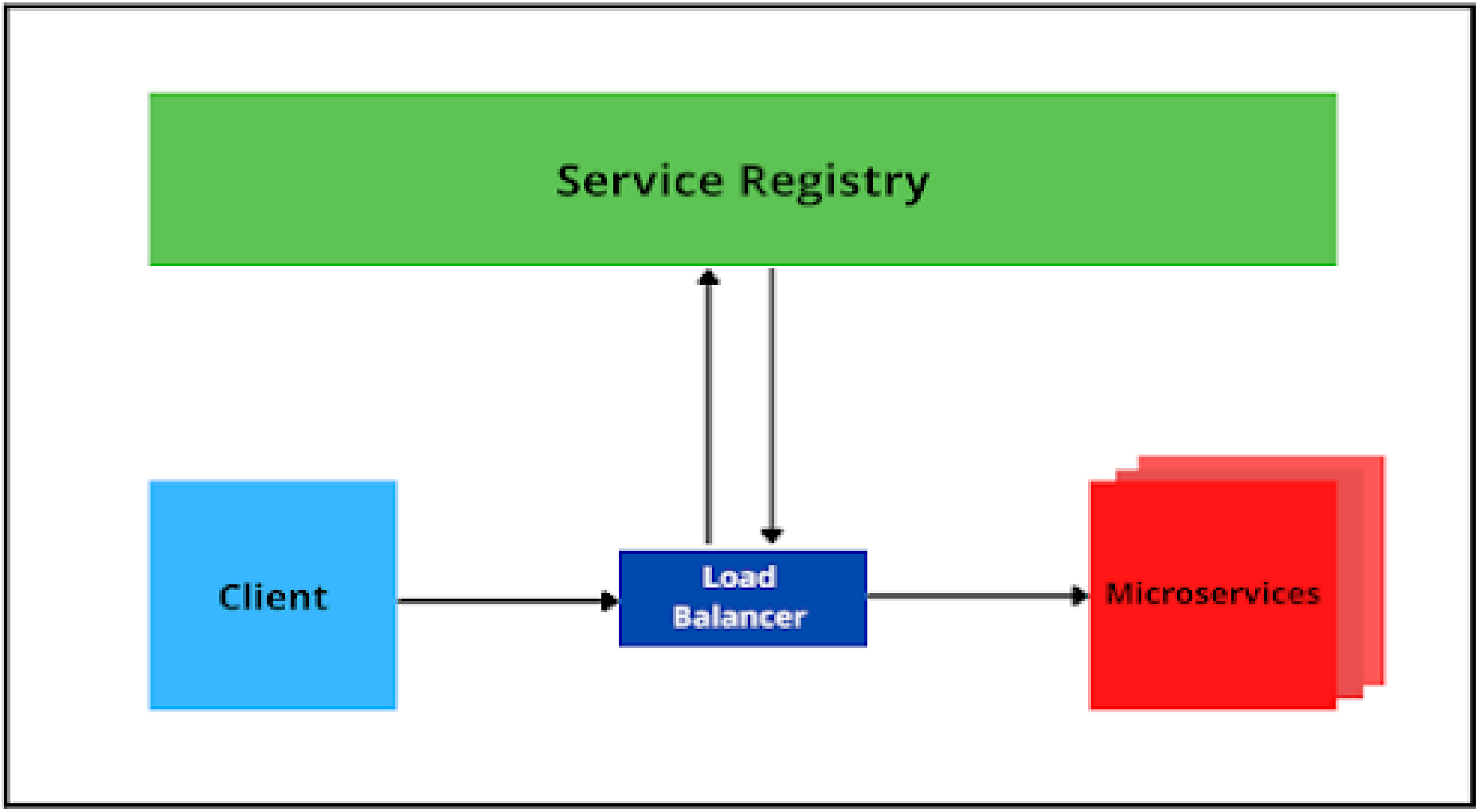


Config Server Architecture

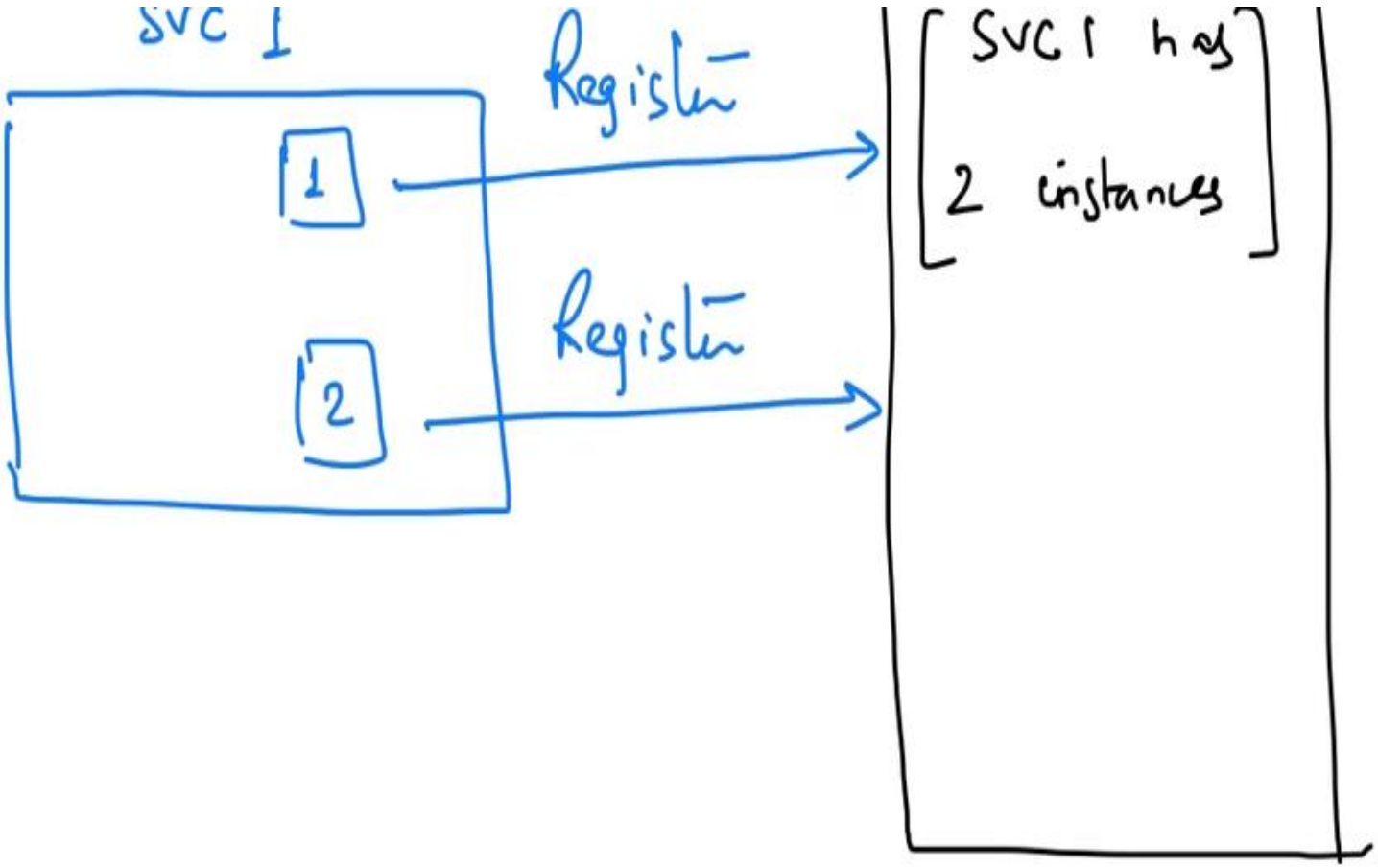
Service Discovery

- **Service Registration**: The process where a service instance notifies the Service Registry of its network location (IP, port) and metadata upon starting up.
- **Service Discovery:** The process where a client (service) queries the Service Registry to find the available network locations of another service it needs to talk to.
- **Service Registry:** The centralized database (e.g., Netflix Eureka, Consul, Nacos) that stores this information.
- **Service discovery** is the **process** of automatically detecting devices and services on a computer network. This reduces the need for manual configuration by users and administrators.
- **Service registration** and **Service discovery** are two closely related parts of the same mechanism, often managed by a single tool like Netflix Eureka. Registration is the action of a service adding its location to a database upon startup, while Discovery is the process of finding that location to make a request.

Diagrammatic Representation



contd..



How They Work Together

- **Start-up (Registration):** When "Service A" starts, it tells the Registration Service, "I am here, my IP is X".
- **Lookup (Discovery):** When "Service B" needs to call "Service A", it asks the Discovery Service, "Where is Service A?".
- **Communication:** The Discovery Service provides the location, and "Service B" calls "Service A" directly.
- **Why Separate Them?**
  - ✓ **Dynamic Environments**: In cloud environments, services scale up, down, or fail constantly. Registration updates the registry in real-time.
  - ✓ **Decoupling:** Services don't need to know the hardcoded IP addresses of other services; they only need to know the location of the Discovery Service.
  - ✓ Common implementations of this pattern include Netflix Eureka, HashiCorp Consul, or Alibaba Nacos.

Types of Service Discovery

- There are two types of Service Discovery
  a. Client-Side Service Discovery
  b. Server-Side Service Discovery
- Point to Remember:
  a. Client-Side Service Discovery Example: Netflix Eureka, Zookeeper, Consul
  b. Server-Side Service Discovery Example: NGNIX, AWS ELB

Feign Client

- **Feign Client** (specifically Spring Cloud OpenFeign) is a **declarative REST client** used to simplify communication between microservices.
- **Interface** is defined with annotations, and implementation is provided at runtime instead of writing manual code to build HTTP requests and handle responses e. with RestTemplate or WebClient.
  - ✓ **Declarative Programming:** The API rendering is defined using interface and standard annotations viz.. @GetMapping or @PostMapping. Spring handles the "how" of making the network call.
  - ✓ **Reduced Boilerplate:** It eliminates repetitive code for building URLs, setting headers, and serializing/deserializing JSON data.
  - ✓ **Seamless Integration:** Built to work within the SpringCloud ecosystem integrating natively with:
    - ❑ **Service Discovery:** Works with Netflix Eureka to resolve service names into physical IP addresses.
    - ❑ **Load Balancing:** Automatically distributes requests across multiple service instances using Spring Cloud LoadBalancer.
    - ❑ **Fault Tolerance:** Integrates with Resilience4j or Hystrix to provide fallbacks and circuit breaker patterns.

contd..

- **How to Implement a Feign Client – t**o use Feign follow these standard steps:
  - ✓ **Add Dependencies:** Include the OpenFeign Starter in your pom.xml or build.gradle.
  - ✓ **Enable Feign Clients:** Add the @EnableFeignClients annotation to your main application or configuration class.
  - ✓ **Define the Client Interface:** Use the @FeignClient annotation on an interface.
- While RestTemplate is in maintenance mode, the Spring Documentation recommends using **WebClient** for reactive/non-blocking calls and **OpenFeign** for clean, synchronous microservice communication.

Comparative table

Feature	Feign Client	RestTemplate
Approach	Interface-driven (Declarative)	Template-driven (Imperative)
Boilerplate	Low (Automatic)	High (Manual URL/JSON handling)
Maintenance	Easier to read and update	Can become complex and repetitive
Integration	Native Spring Cloud support	Requires manual configuration for Load Balancing

contd..

- Notable API Gateways
- Netflix API Gateway: Zuul
- Amazon API Gateway
- Kong API Gateway
- Apigee API Gateway
- MuleSoft

Potential Drawbacks:

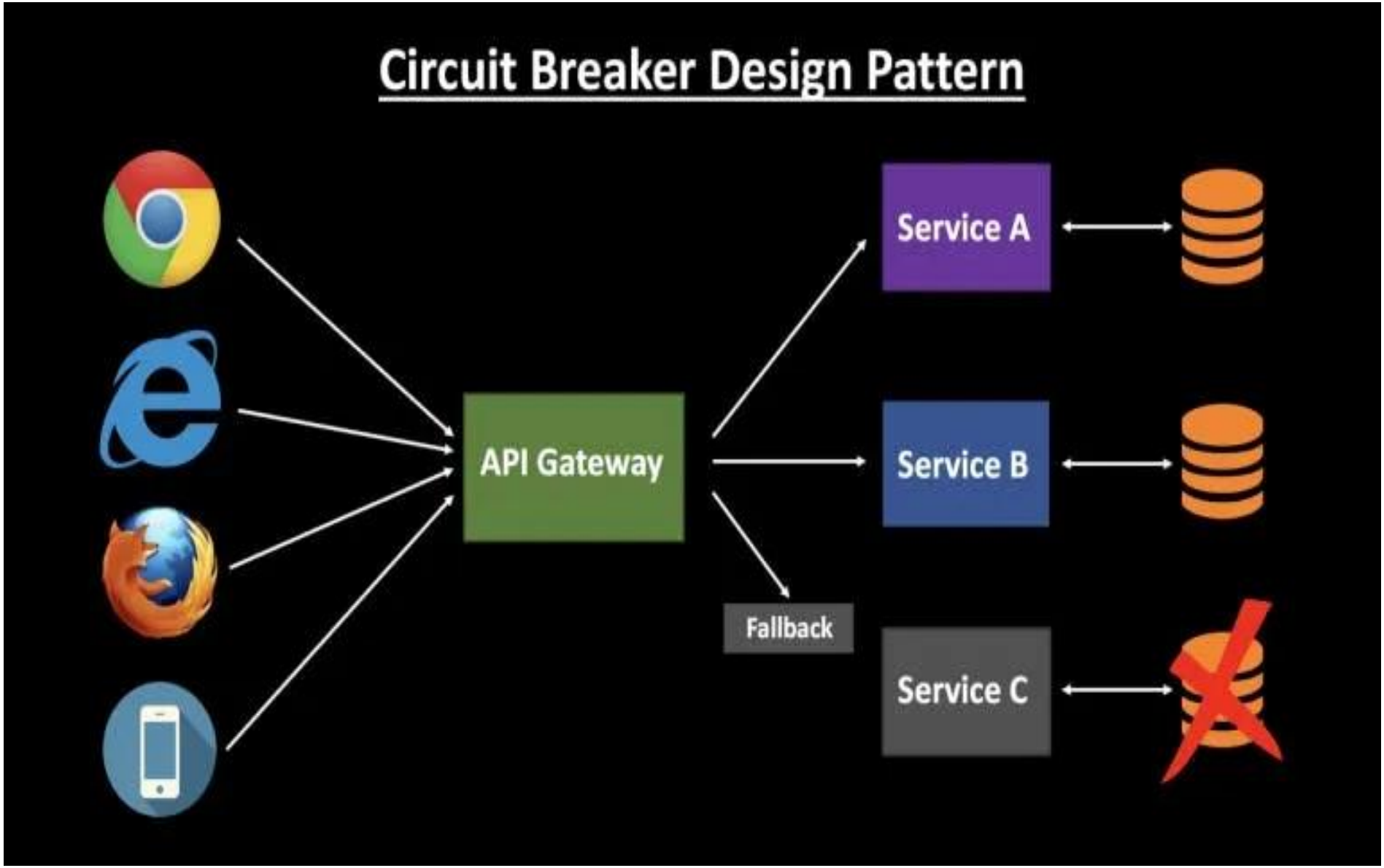
•**Single Point of Failure:** If the gateway goes down, the entire system can fail.

•**Potential Bottleneck:** High-volume traffic can overwhelm the gateway if not scaled properly.

Resilience & fault tolerance

- **Fault tolerance** means given a microservice application how much tolerant it is or what is the impact when there is a fault, will the whole application goes down or part of functionality goes down or there are some arrangements to handle failures.
  Tolerance the system has for a fault — Fault Tolerance
- **Resilience** means how many faults the system can tolerate.
- **Fault tolerance** and **resilience** in microservices ensure systems remain functional despite component failures or network issues, preventing cascading outages.
- Key strategies include isolating services, implementing circuit breakers to stop calls to failing components, using retries for transient errors, and setting strict timeouts. These techniques maintain system availability and gracefully degrade functionality rather than crashing completely.

Diagrammatic Representation



Key Techniques for Resilient Microservices

- **Circuit Breaker Pattern:** Detects failures and prevents the system from constantly trying to call a failing service, allowing it to recover.
- **Bulkhead Pattern :** Partitions system resources (e.g., thread pools) into isolated pools so that if one service fails, others continue to function.
- **Retry Pattern:** Automatically retries a failed request, useful for transient network glitches, but should be used with limits to avoid overloading services.
- **Timeout Pattern:** Stops waiting for a response after a set time, preventing a slow service from cascading delays throughout the system.
- **Fallback Methods:** Provides a default response or alternative functionality when a service call fails.
- **Service Isolation:** Ensuring services are independent prevents a failure in one service (e.g., ordering) from crashing another (e.g., user profile)

contd..

- When building microservices, it is crucial to design with the expectation that failures will occur. This proactive approach involves planning and building systems that can gracefully handle failures and recover with minimal impact.
- **Self-Healing Mechanisms**
- Self-healing mechanisms are another critical component of fault tolerance. These mechanisms allow services to recover from failures without external intervention.

Observability & Monitoring

- Observability and monitoring in microservices are essential, complementary practices for managing distributed system health. **Monitoring** tracks predefined metrics (e.g., CPU, latency) to tell you *if* a system is failing. **Observability** uses logs, metrics, and traces to explain *why* it is failing by providing deep insights into internal states across complex service dependencies.

Annotations

1. @EnableConfigServer

This Spring Cloud annotation turns application into a server that other apps can use to get their configuration. This is a very useful annotation using Spring Cloud, user can have one dedicated Java service for Configuration.

2. @EnableEurekaServer

This Spring Boot annotation makes the application a Eureka discovery service, which other applications can use to locate services through it. This is an important step in developing microservices in Java using Spring Cloud.

3. @EnableDiscoveryClient

This Spring Cloud annotation makes an application register in the service discovery and discovers other services through it.

contd..

4. @LoadBalanced

Used as a marker annotation indicating that the annotated Rest Template should use a RibbonLoadBalancerClient for interacting with the service(s).

5. @EnableCircuitBreaker

This Spring Cloud annotation configures Hystrix circuit breaker protocols. Use of the Circuit Breaker pattern allows a microservice to continue operating when a related service fails, preventing the cascading failure and thus giving the failing service time to recover.

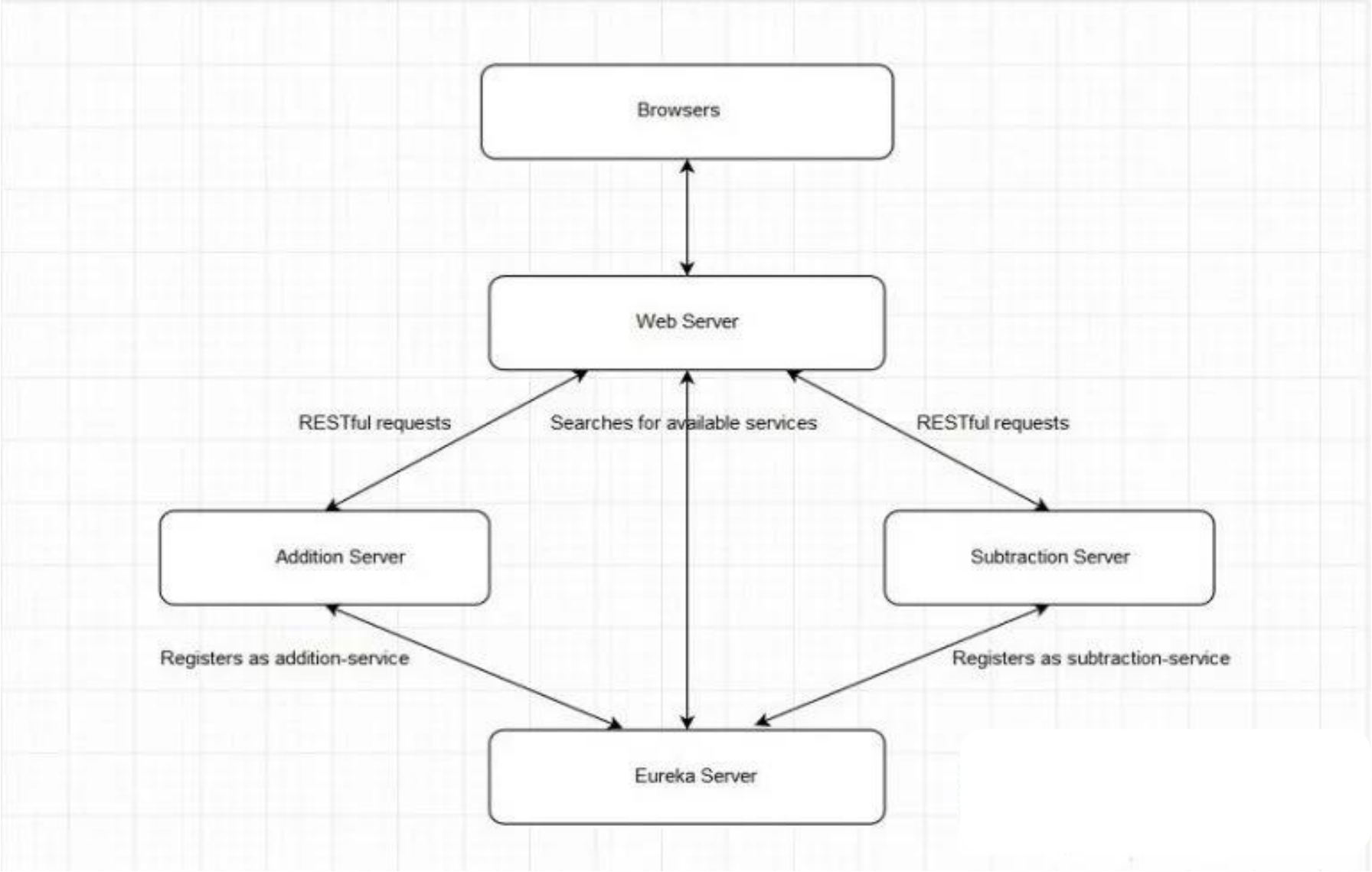
6. @HystrixCommand(fallbackMethod = "fallbackMethodName")

This Spring boot annotation marks methods to fall back to another method if they cannot succeed normally.

Create a MicroService Application

- Step 1 :
- Create a Spring boot application and add the annotation @EnableEurekaServer
- By default, each Eureka Server is also a Eureka client and needs at least one service URL to locate a peer.
- The entire application will have a single Eureka Server node (Standalone Mode), hence disable its client-side behavior.
- Its done by configuring the properties in the *application.properties* file.

Use case – Calculator system

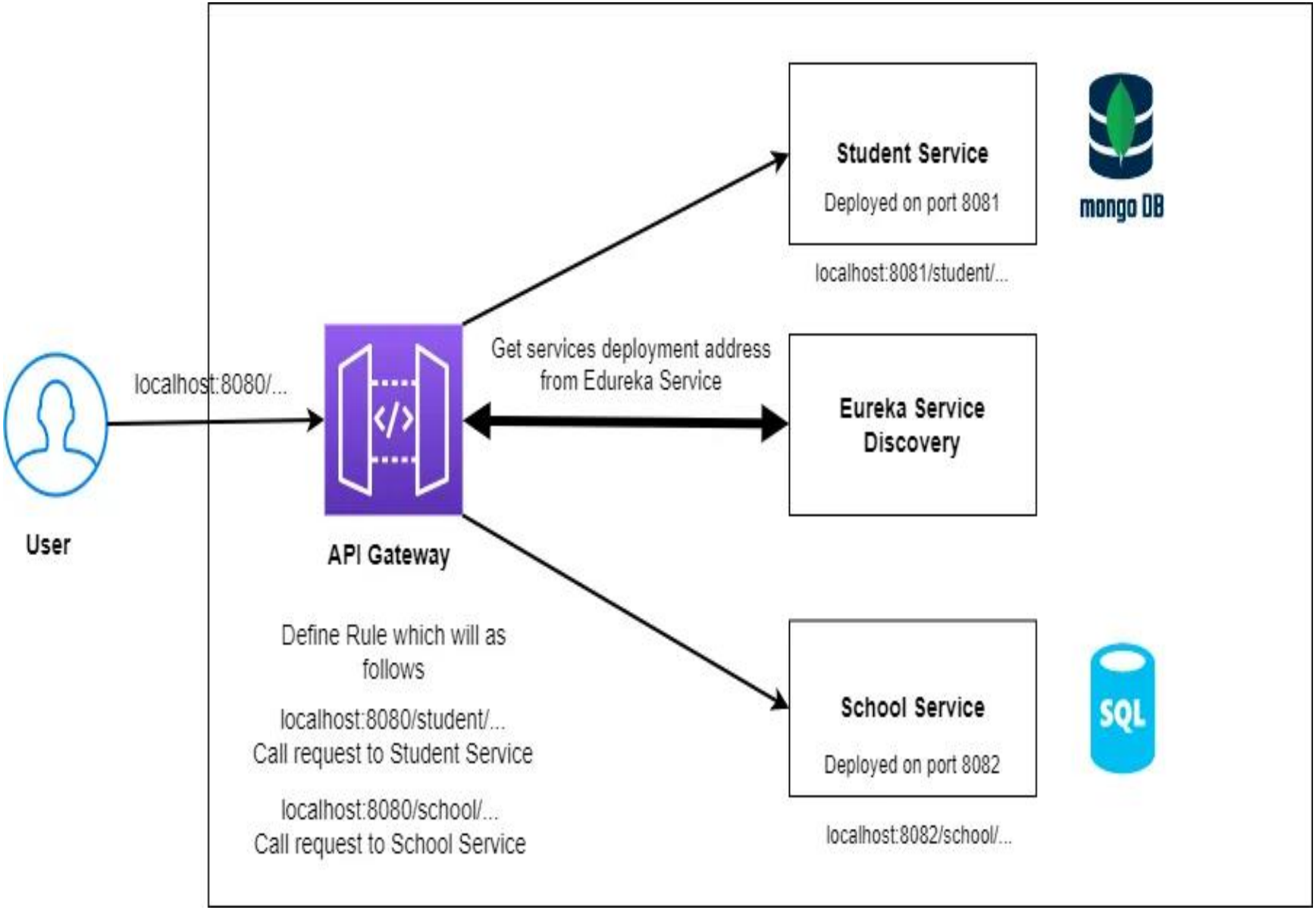


contd..

- This web application an arithmetic service. An arithmetic calculator served in a microservice approach.
- The diagram shows two microservices, **Addition Server** and **Subtraction Server**.
- The servers need to find each other so they need to register to the discovery server called **Eureka**.
- Once the **Web Server** finds the micro service, it can then send requests,i.e. sends RESTful requests.
- When the Web Server receives the response it then serves the result to the browser.

Load balancing in Microservices

- **Load balancing** is the process of distributing traffic among different instances of the same application. Multiple instances of each application are run and whenever one service needs to communicate with another, it needs to pick a particular instance to send its request.
- **Load balancing** is needed to create a fault-tolerant system, to achieve high availability, scalability, and efficiency.
- **Load balancing algorithms** are the rules or methods that determine how to distribute the requests among the service instances. **Round robin** is a widely used algorithm that assigns requests to service instances in a circular order without considering other factors. **Random**, a variation of round robin, assigns requests randomly without considering other factors. **Least connections** assigns requests based on the number of active connections, while **weighted** assigns requests based on predefined weights that reflect the relative capacity or performance.


Student Service
Deployed on port 8081
mongo DB
localhost:8081/student/...
Get services deployment address from Edureka Service
localhost:8080/...
Eureka Service Discovery
User
API Gateway
Define Rule which will as follows
localhost:8080/student/...
Call request to Student Service
localhost:8080/school/...
Call request to School Service
School Service
Deployed on port 8082
SQL
localhost:8082/school/...

Use case - 2

- There are Student microservice and School microservice. student service has a NoSQL database called MongoDB and School service uses a SQL database.
- The Edureka Service Discovery – Service Discovery is a pattern where client applications can discover the location of services dynamically at runtime. The main idea is to have a central registry of all the services and their current status. This registry can be queried by clients to determine the location of a particular service.
- **API Gateway** – An API Gateway is a reverse proxy that sits in front of all microservices and acts as a single entry point for incoming API requests. It is responsible for request routing, composition, and protocol translation, among other things.

contd..

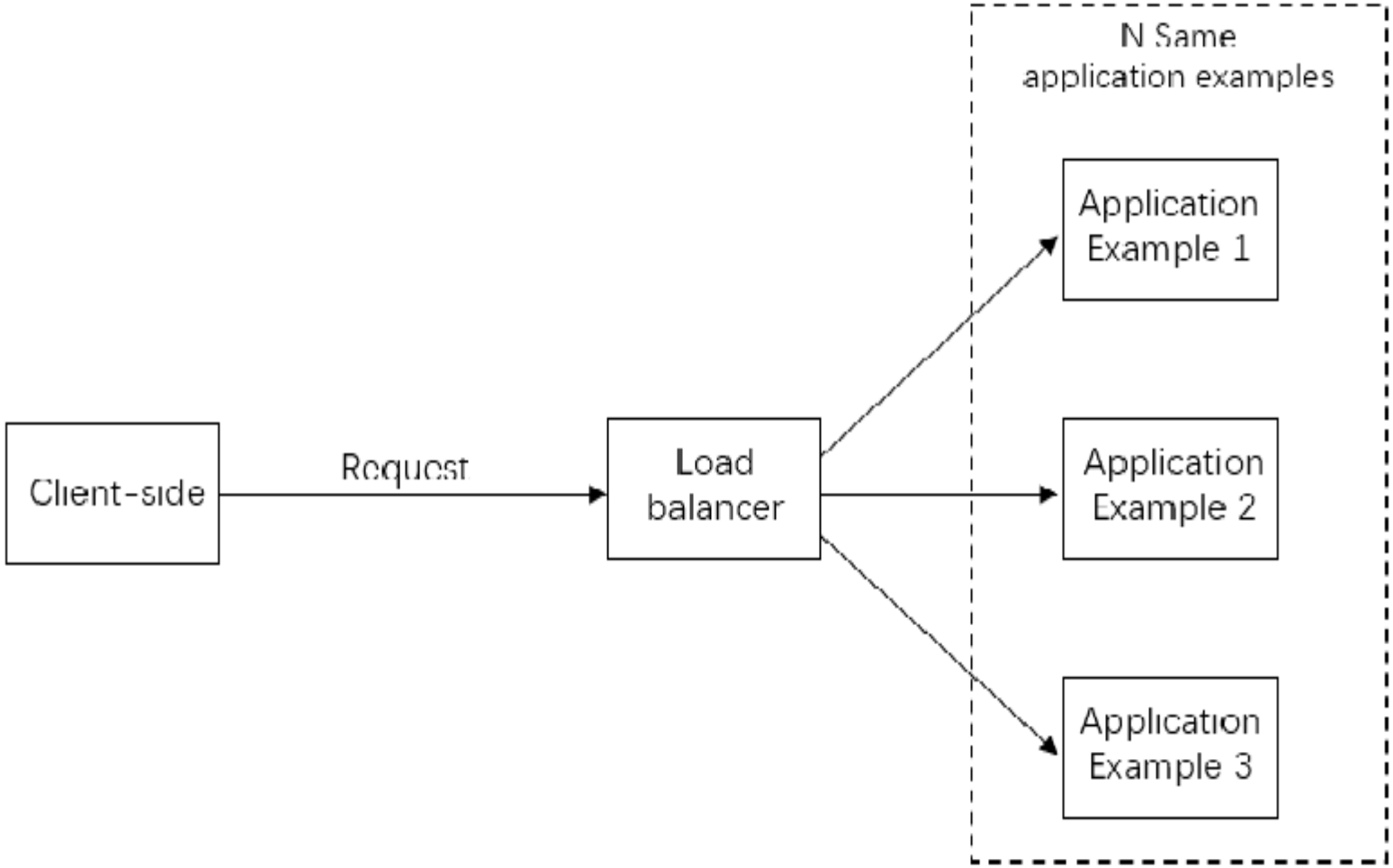
- The connection between Service Discovery and API Gateway is that the API Gateway uses the information from the Service Discovery registry to route requests to the appropriate microservice.
- The API Gateway queries the registry to determine the location of the target microservice, and then forwards the incoming request to that service.
- In this way, the API Gateway acts as a bridge between client applications and the microservices, providing a unified entry point for incoming requests and abstracting the underlying complexity of the microservices.

contd..

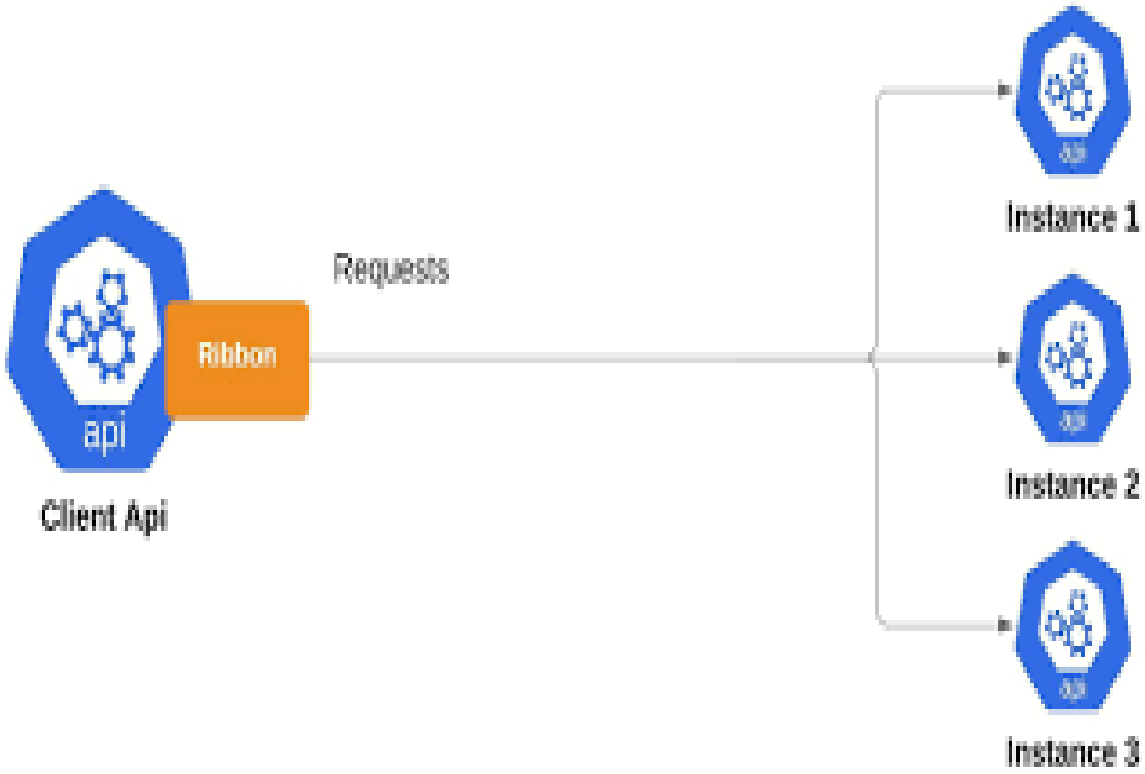
- **Load balancing strategies –**

1. **Client-side** load balancing involves the client, such as another service or external application, selecting which service instance to send the request to, based on criteria or algorithm. This allows more control and flexibility, but requires additional logic and coordination.
2. **Server-side** load balancing requires the server, such as a dedicated load balancer or gateway, to choose which service instance to send the request to. This simplifies client logic and reduces network hops, but also creates a single point of failure and potential bottleneck.
3. **Hybrid** load balancing combines both client-side and server-side load balancing, where the client and server cooperate to distribute the load. This approach can benefit from both approaches, but also increases complexity and overhead.

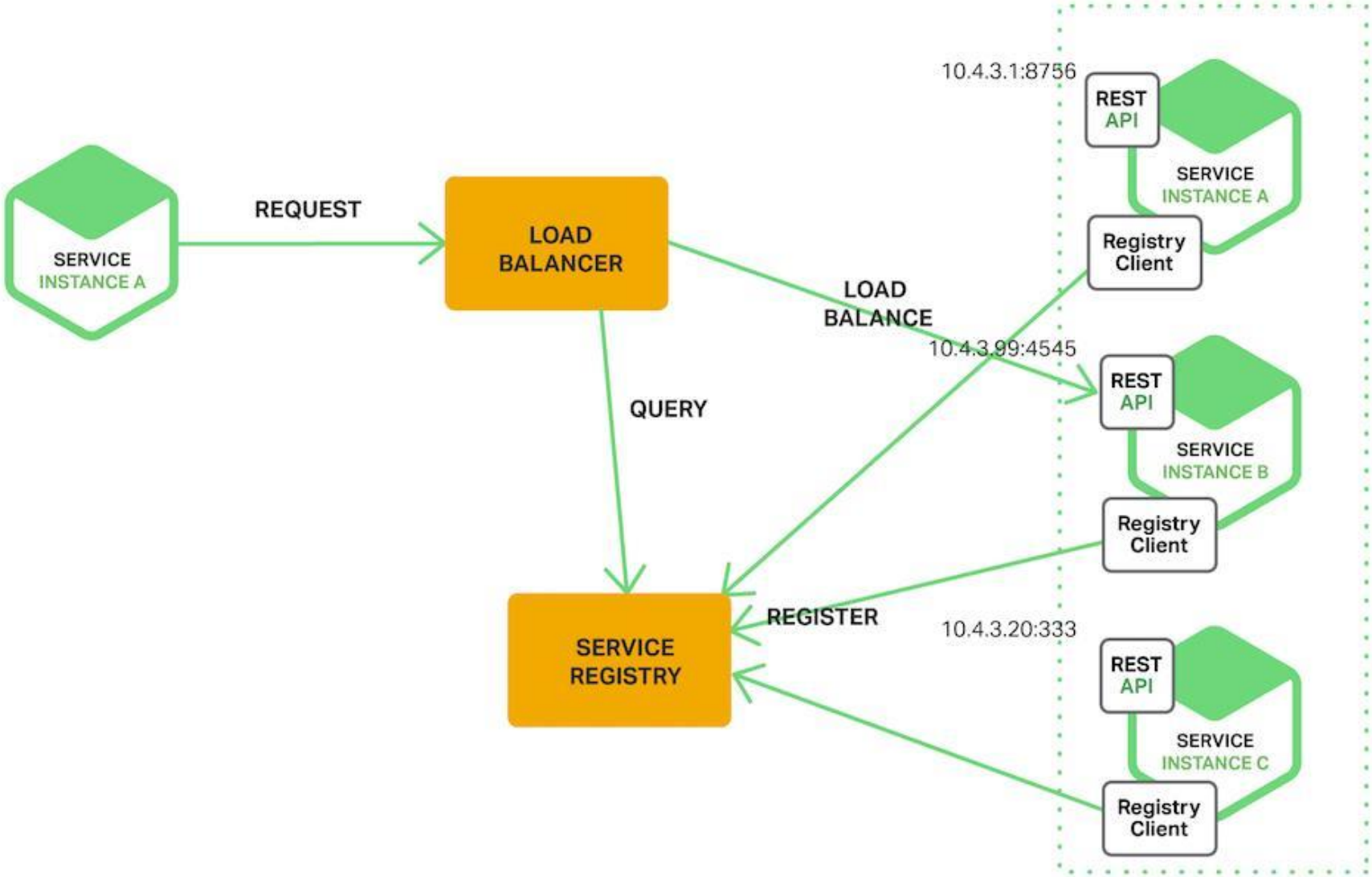
Diagrammatic Representation



contd..


Requests
Ribbon
api
Client Api
Instance 1
Instance 2
Instance 3

contd..


SERVICE
INSTANCE A
REQUEST
LOAD
BALANCER
QUERY
LOAD
BALANCE
10.4.3.1:8756
REST
API
SERVICE
INSTANCE A
Registry
Client
10.4.3.99:4545
REST
API
SERVICE
INSTANCE B
Registry
Client
REGISTER
SERVICE
REGISTRY
10.4.3.20:333
REST
API
SERVICE
INSTANCE C
Registry
Client

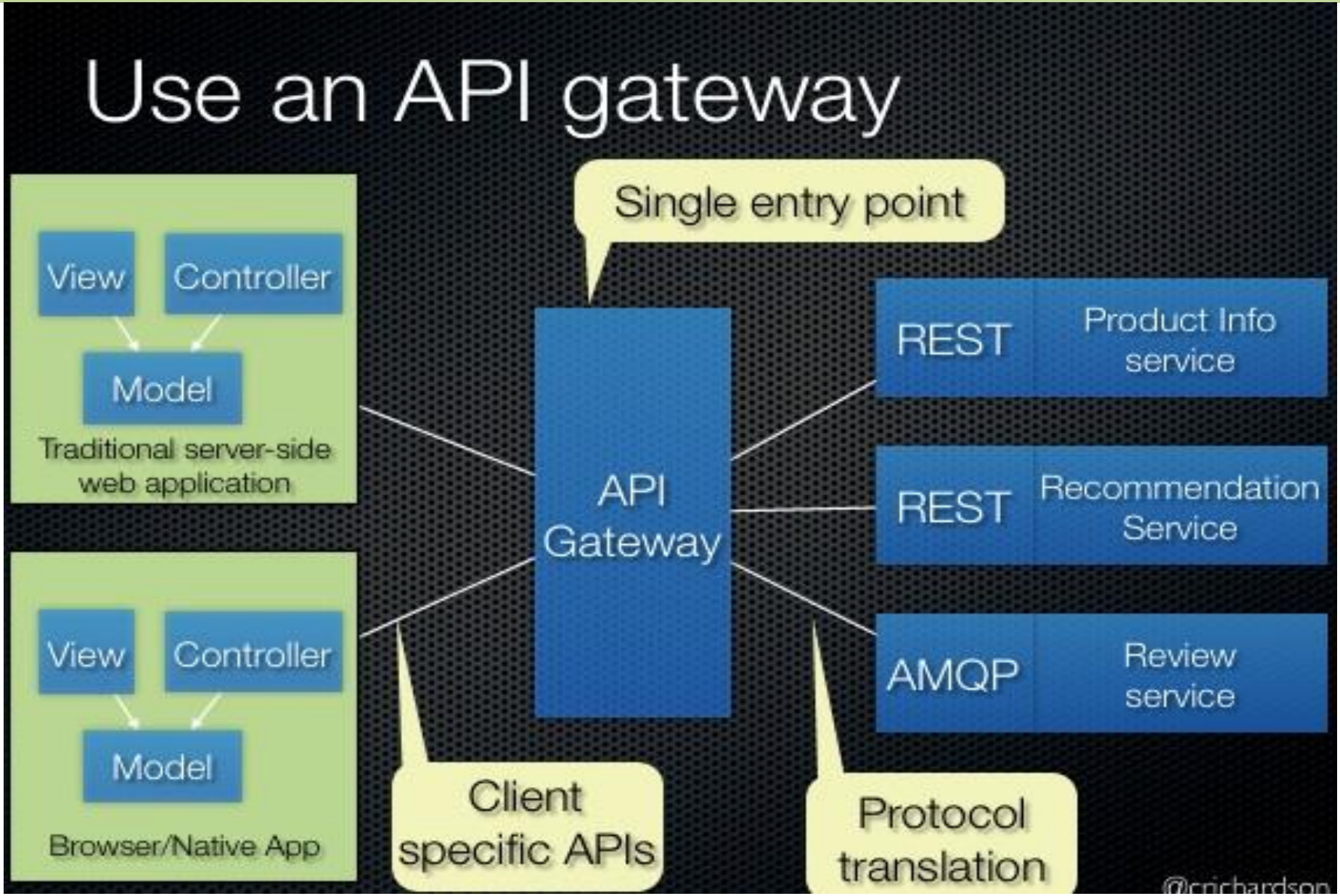
contd..

- **Load balancing tools** – are the software or hardware components that implement the load balancing strategies and algorithms.
- **Load balancer** is a dedicated device or server that acts as a proxy between the clients and the service instances, and distributes requests according to an algorithm.
- Depending on their scope and functionality, there are various types of load balancing tools available, such as a load balancer, service mesh, or API gateway.
- Service meshes use sidecar proxies that run alongside each service instance and a control plane to manage communication and coordination among microservices. API gateway is a single entry point that exposes the microservices to external clients, providing features such as authentication, authorization, rate limiting, logging, or caching. It can also perform load balancing by delegating to a load balancer or by implementing its own algorithm.

API gateway

- **API gateway** is a software between a client and a set of backend microservices, handling multiple API calls simultaneously and route them to different backend services. Also decompose a single client call into multiple requests to other microservices and aggregate the results when they respond. Works as **reverse proxy**.
- **API gateway** can **load balance** between replicas of the same service, which allows better resource utilization. Sophisticated algorithms are used – weighted connections, least connections, or even custom implementations that leverage the service registry.
- **API Gateway** can **translate between protocols**, allowing clients to call any service with a single protocol. API Gateways effectively hide the service implementations from client applications. As long as the interface definition remains the same, developers can implement changes to the service logic using any technology stack without the client knowing it.

contd..


Use an API gateway
Single entry point
View
Controller
Model
Traditional server-side web application
View
Controller
Model
Browser/Native App
API Gateway
REST
Product Info service
REST
Recommendation Service
AMQP
Review service
Client specific APIs
Protocol translation
@crichardson

contd..

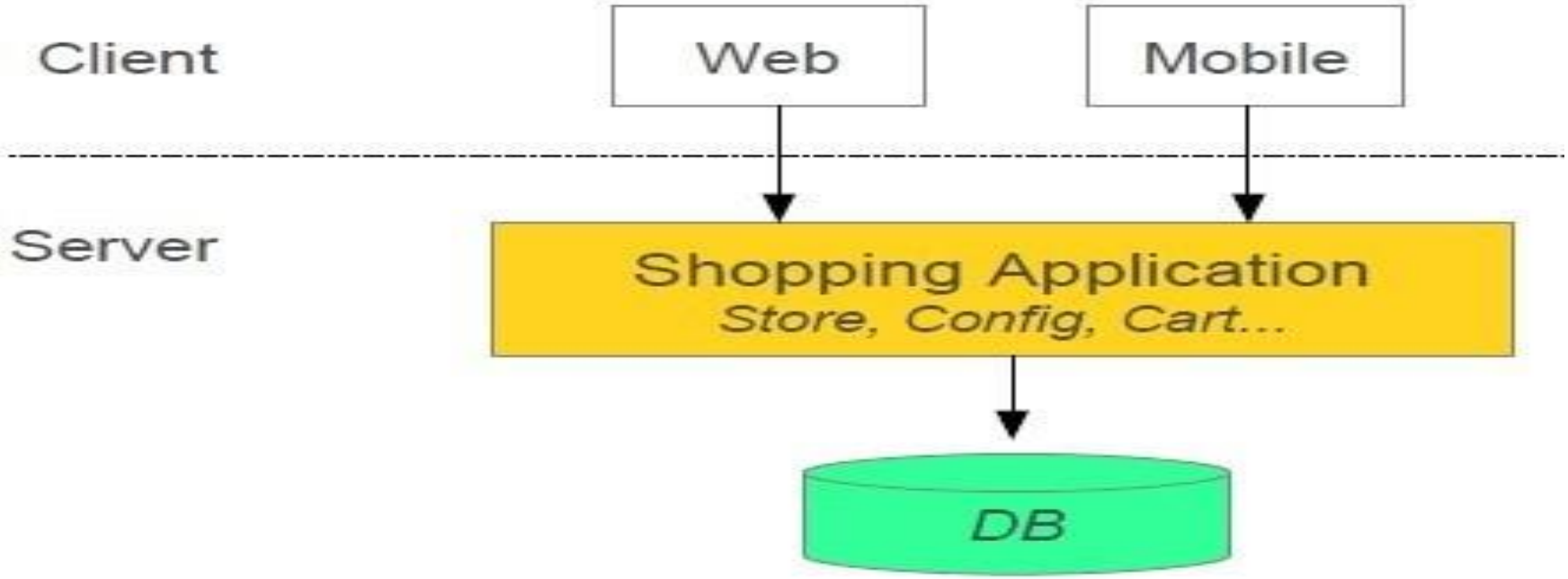
- **API gateways** are an effective barrier against Distributed Denial of Service (DDoS) attacks—throttling the number of requests made to affected services, so the service isn't overwhelmed and protecting it against becoming unresponsive. Request throttling and blacklist capabilities are necessary to keep systems reliable and secure.
- **API Gateway** authenticates all traffic before routing it to the called service. The API Gateway can perform the authentication itself or use external authentication providers for that task.
- **API Gateway** can use its access control lists (ACL) for this authorization or fetch authorization information from an external service provider.
- **API gateways** can be configured to route traffic following deployment strategies canary release, switching or routing traffic between old and new service versions accordingly.

contd..

- **Shopping Cart case study**
- **To build a simple microservices system following steps required**
  1. Creating Discovery Service (Creating Eureka Discovery Service)
  2. Creating MicroService (the Producer)
     - Register itself with Discovery Service with logical service.
  3. Create Microservice Consumers find Service registered with Discovery Service
     - Discovery client using a smart **RestTemplate** to find microservice.

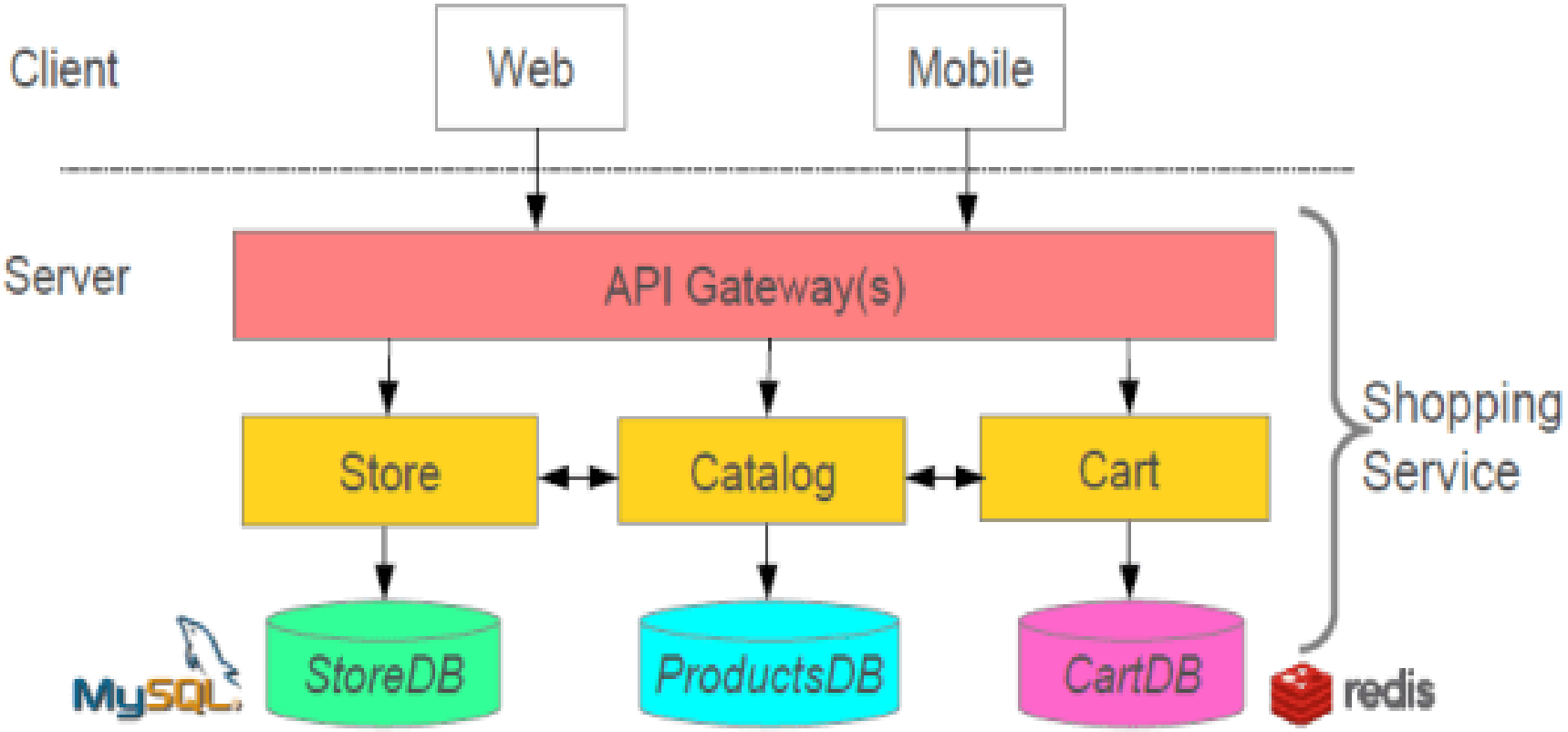
Use case – Shopping cart

- **Shopping cart system without Microservices (Monolith architecture)**
- In this Monolith architecture i.e. all collaborating components combine all in one application.

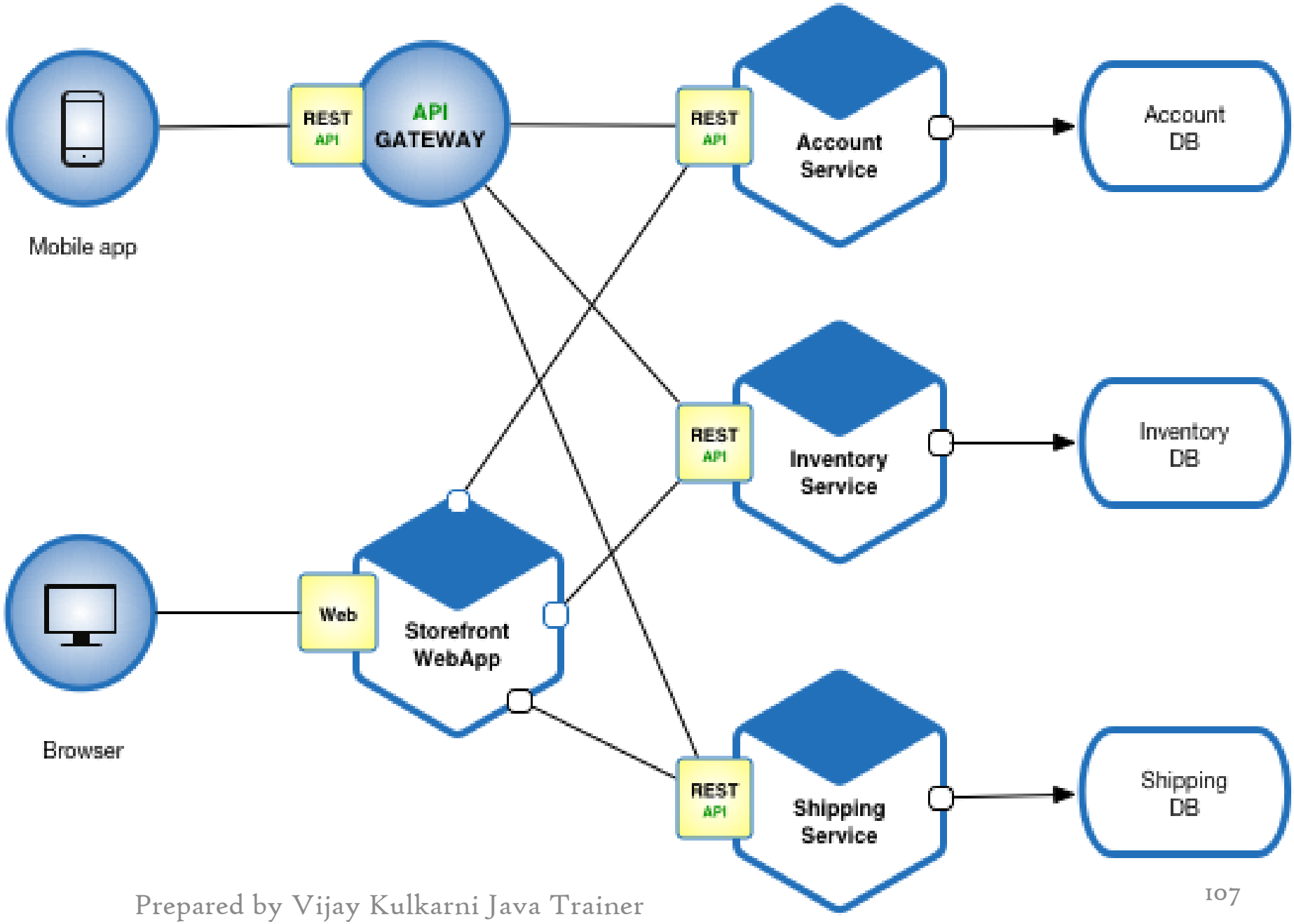


contd..

In MS architecture style, the main application divided into a set of sub-applications collaborating processes called microservices as below.

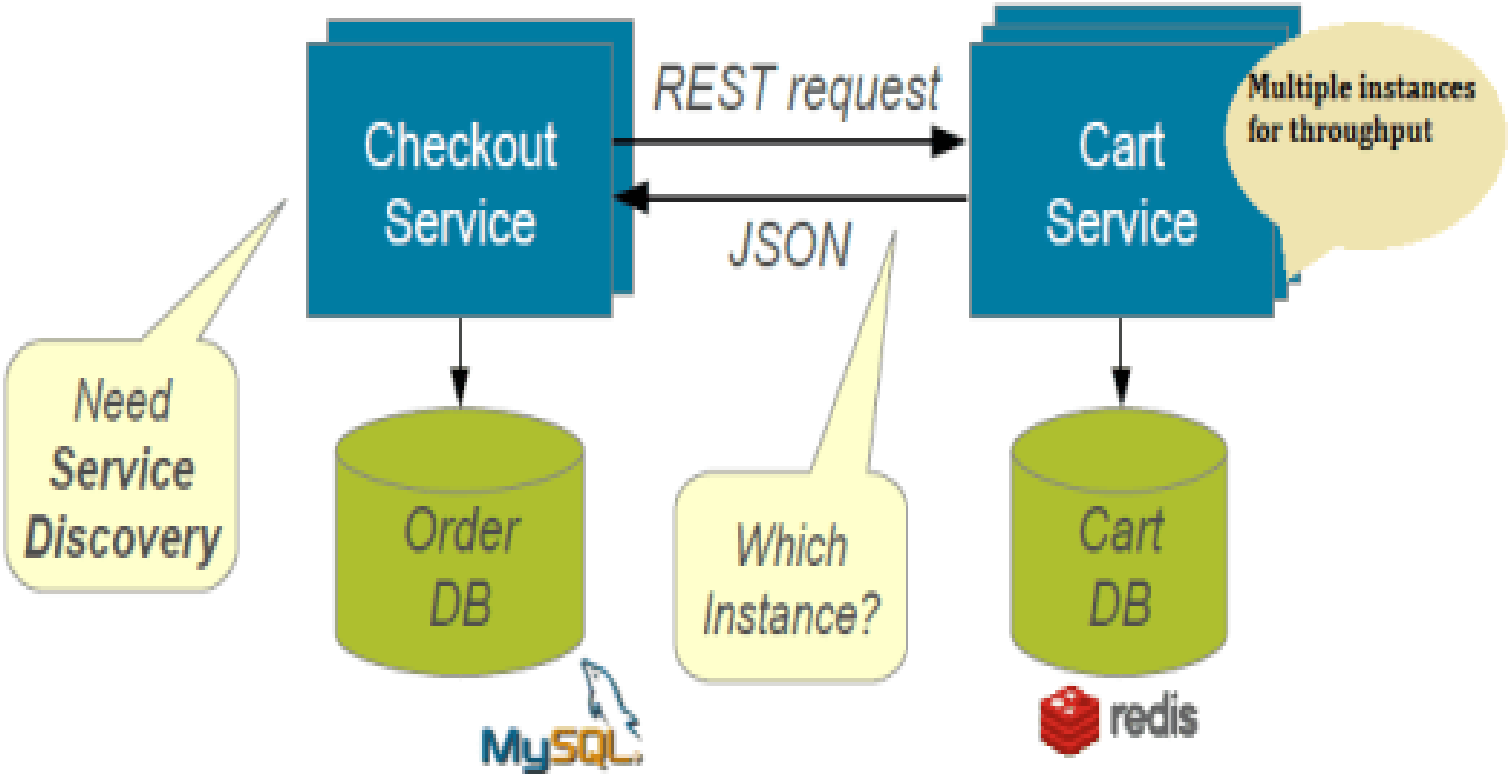


contd..

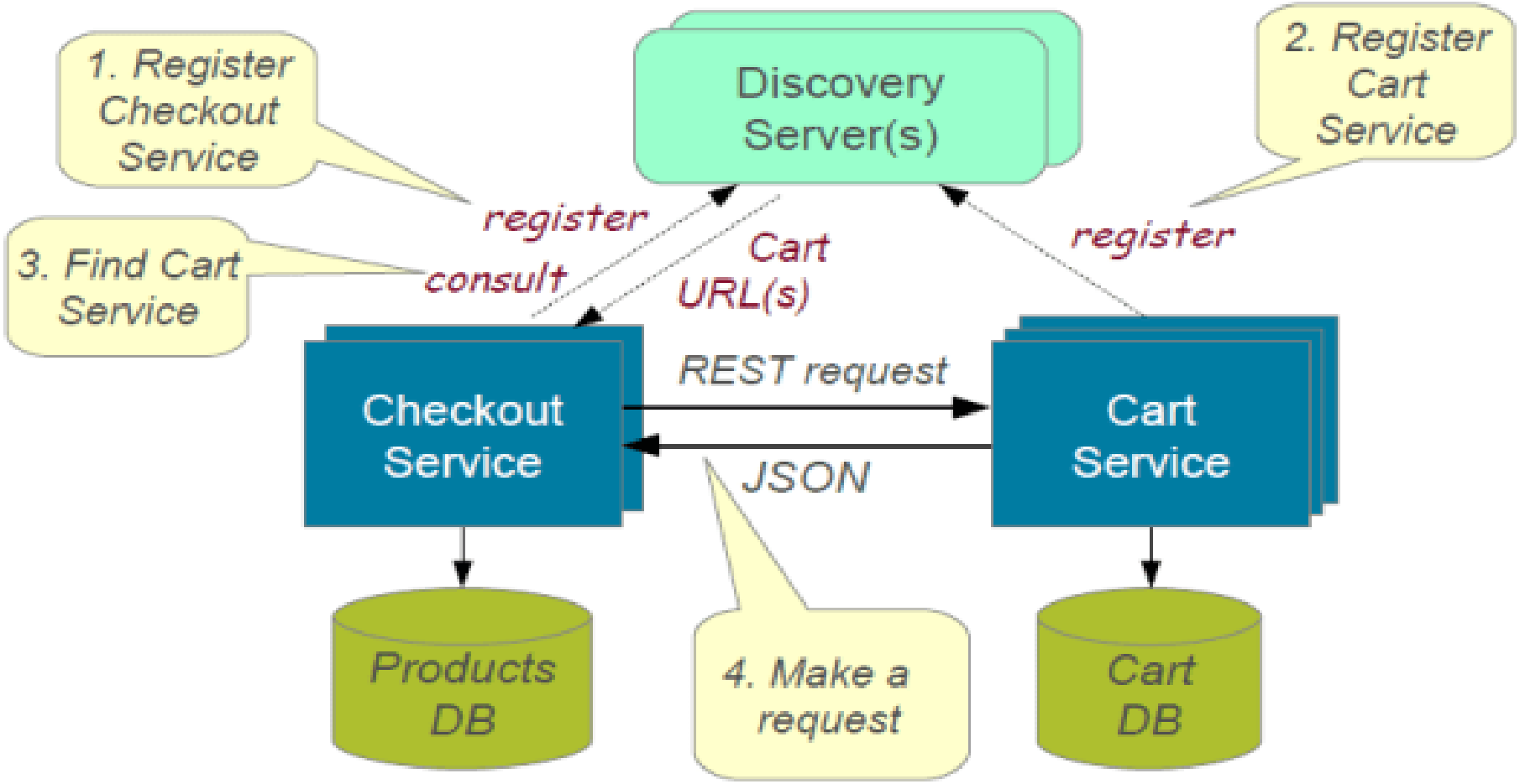

Mobile app
REST
API
API
GATEWAY
REST
API
Account
Service
Account
DB
REST
API
Inventory
Service
Inventory
DB
Browser
Web
Storefront
WebApp
REST
API
Shipping
Service
Shipping
DB

Service Discovery - Problem without discovery

- How do services find each other?
- What happens if we run multiple instances for a service

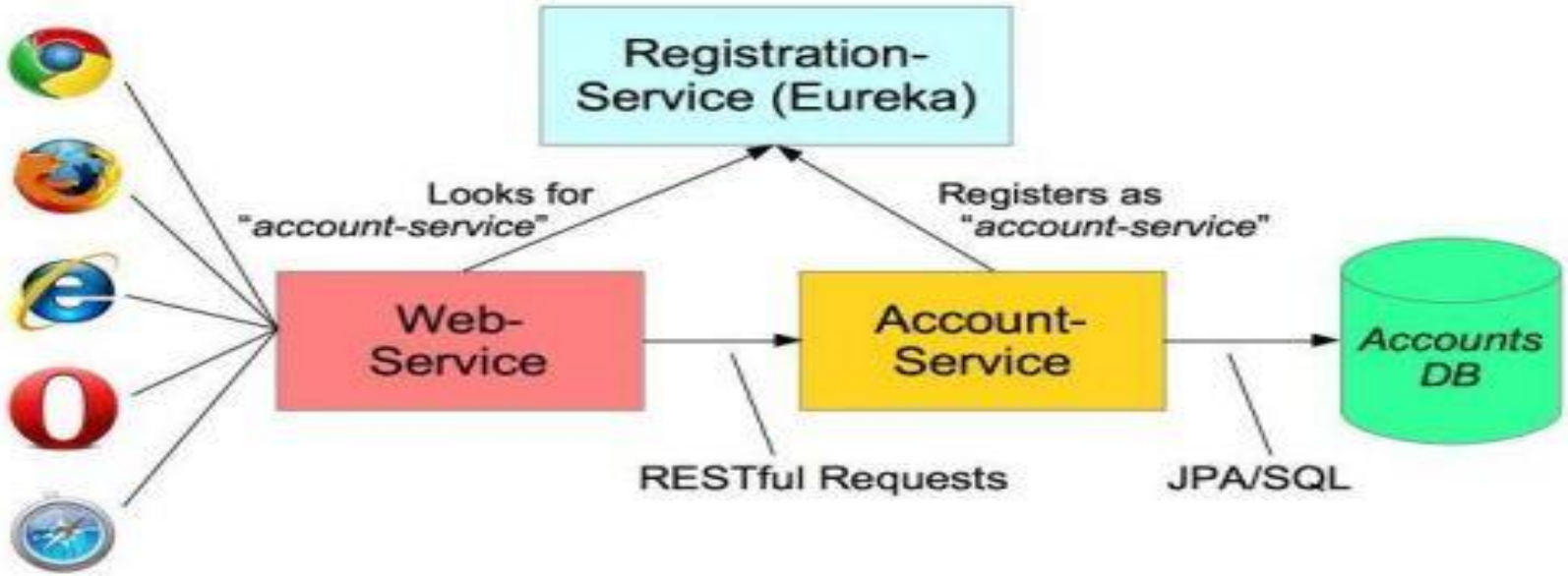


Resolution with service discovery



contd..

- **Implementing Client-Side Load Balancing** – Netflix Ribbon provides several algorithms for Client-Side Load Balancing.
- Spring provides smart **RestTemplate** for service discovery and load balancing by using ***@LoadBalanced*** annotation with **Rest Template** instance.



Challenges to Microservices architecture

- **Quick Setup needed** : Developer should be able to create microservices quickly.
- **Automation :** As a number of smaller components are involved, its imperative automate - Builds, Deployment, Monitoring etc.
- **Visibility :** Needs large and auto monitoring as large number of smaller components are deployed and maintained.
- **Bounded Context :** Microservice boundaries get evolved, so bounded contexts from domain design can be the start point.
- **Cost** : Microservice is costly, as it requires to maintain different server space for different business tasks.
- **Configuration Management :** Hundreds of configurations exist across application, hence need for config management.

contd..

- **Consistency :** It is important to have some decentralized governance around the languages, platforms, technology and tools.
- **Dynamic Scale Up and Scale Down :** Microservices can be relished if the applications can scaled up and down easily in the cloud.
- **Pack of Cards :** Microservices should be fault tolerant by Design. If bottom of the chain fails, it can have knock-on effects on others.
- **Debugging :** Centralized logging and dashboards are essential to make it easy to debug problems. As multiple services are involved.
- **Distributed system** : A huge set of skilled professionals are required to support this big heterogeneous distributed software.
- **Enterprise readiness** : It is quite difficult to make a microservice application enterprise ready compared to conventional software development model.

Solutions to Challenges

- Enable building production ready applications quickly
- Provide non-functional features
- embedded servers (easy deployment with containers)
- metrics (monitoring)
- health checks (monitoring)
- externalized configuration

when NOT to use microservice

- When the application size does not justify the need to split it into many smaller components, using a microservices framework may not be ideal.
- There's no need to further break down applications that are already small enough as-is.
- Versioning rules for on-premise microservices are more stringent. Track each individual microservice that participates in a release. Thorough integration and end-to-end testing is, as it can't test in production.
- Troubleshooting a microservice application is substantially more difficult without direct access to the production environment.
- If monolith has some more life, if not yet ready for change then continue.

Thank You